

Directional Visual Perception for Object Sonification in Indoor En- vironments

- A tool for the blind

*Visuell Riktningsuppfattning för Objektsonifiering i Inomhus-
miljöer*
- Ett redskap för blinda

Edvard Nilsson

Supervisor : Arvind Balachandran
Lars Nielsen
Examiner : Per-Erik Forssén

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: <http://www.ep.liu.se/>.

Abstract

It is a well known fact that physical activity and sports provide significant health benefits. While many different assistive tools have been developed for visually impaired individuals throughout the years, they still often lack the assistive technology to participate in sports independently. In this thesis, the gap is addressed through the development of a prototypical platform that utilizes computer vision and machine learning to create directional perception for the sonification of objects in real-time. The system, built up of a Raspberry Pi 5 and an AI Camera, employs the pinhole camera model to translate 2D pixel coordinates of an object into a 3D target vector consisting of an azimuth, elevation and distance. These vectors are then rendered into spatial audio with Head-Related Impulse Responses (HRIR), and a sound is played inside a pair of headphones that corresponds to the direction of said object, along with a frequency attenuation dependent on the distance to it.

A comparative analysis was conducted between two object detection algorithms in this context: MobileNet SSD V2 320x320 and Color Segmentation (ColSeg). The results demonstrated that MobileNet SSD maintained exceptional hardware stability, averaging a 20% CPU load with zero dropped targets over a one-hour stress test. This was possible due to the AI Cameras internal IMX500 sensor handling the image processing. However, MobileNet's object detection provided inconsistent bounding-box dimensions around human targets under movement, which led to unreliable distance estimates.

Conversely, while ColSeg provided stable distance metrics for rigid objects like balloons, its processing demands over a one-hour stress test caused the Raspberry Pi to reach its 85°C thermal throttling limit. This resulted in a cascading hardware failure: HRIR calculation latency spiked over 30 ms, causing buffer backlogs, target drops, and severe audio stuttering.

The study concludes that while the developed platform establishes a robust foundation for a non-invasive directional perception tool, relying on 2D bounding boxes for depth estimation is methodologically insufficient for dynamic sports environments. To ensure system survivability and user safety in future iterations of the system, the inclusion of hardware depth sensors, such as LiDAR or stereo cameras, and the use of pre-computed HRIR filters are recommended.

Acknowledgments

I would like to thank my examiner Per-Erik Forssén, as well as supervisors Arvind Balachandran and Lars Nielsen for the guidance they have given me throughout this project. I also want to thank them and Linköping University as a whole for providing the equipment and hardware which allowed me to build the system. Finally, I want to thank concurrent master student Simon Hansson for providing the code and functionality for the HRIR calculation, as well as his help when questions about it arose.

Contents

Abstract	iii
Acknowledgments	iv
Contents	v
1 Introduction	1
1.1 Motivation	1
1.2 Aim	2
1.3 Research questions	3
1.4 Delimitations	3
2 Theory and Hardware	4
2.1 Definitions	4
2.2 Hardware	5
2.3 Object detection methods	7
2.4 Image projection	8
2.5 Image Distortion	8
2.6 Camera Calibration	10
2.7 Coordinate mapping from a 2D pixel point to a 3D target vector	10
3 Related Work	12
3.1 The SeeHear system	12
3.2 Morphological segmentation of sport scenes	13
3.3 You Only Look Once	13
3.4 MobileNetV2	14
4 Method	15
4.1 Hardware configuration	15
4.2 Calibration	15
4.3 Directional perception	18
4.4 Sound playback	18
4.5 Experiments and tests	18
5 System Description	23
5.1 System overview	23
5.2 Directional perception	23
5.3 Audio playback	26
6 Results	27
6.1 Calibration	27
6.2 Directional perception	32

7	Discussion	46
7.1	Results	46
7.2	Method	48
7.3	The work in a wider context	49
8	Conclusion	51
8.1	Future work	52
	Bibliography	53
A	Appendix	56
A.1	Terminal commands	56
A.2	Code	56
A.3	Tables	58
A.4	Calibration results with default lens	59
A.5	Calibration results with wide lens	60



1 Introduction

This report showcases a system that uses object detection methods to create directional perception for sonification of visual data, along with the succeeding analysis of different object detection methods within this context.

1.1 Motivation

Sports and physical activity (PA) are broadly known to be beneficial for both body and mind [1], [2]. PA and sports have been shown to increase the efficacy of many basal bodily functions, such as improving blood pressure regulation and hippocampus functionality. In addition to this, they also reduce the risk of developing numerous disorders and disabilities, such as cardiopulmonary and neurodegenerative diseases. They also help to reduce the risk of many psychological disorders related to anxiety and depression. Even more than this, PA has been shown to sharpen the senses through increased plasticity of the nervous system [3]. One of the most important senses in sports is vision; necessary when following a ball, identifying obstacles, or differentiating between teammates and opponents. Unfortunately, not everyone has the same conditions when it comes to eyesight. According to the World Health Organization, “at least 2.2 billion people have a near or distance vision impairment” [4]. There is no reason why blind people should be excluded from the objectively positive benefits of PA.

To help visually impaired people benefit from PA and participate in sports, a number of tools and methods have been fashioned through the years. As of writing this thesis, most of the equipment that exists for visually impaired people in sports is analog, if even technical, at best. The types of tools range from footballs emitting sounds to something as simple as tactile markers on equipment in gyms [5], [6].

While sound emitting sport balls are useful for locating the ball itself, the technology to keep track of objects such as opponents, teammates, or environmental obstacles remains limited. In the context of skiing for instance, a whole other person is needed beside the skier to call out for obstacles using a Bluetooth headset [7], or through touch [8]. In 2017 [9], Haladjian et al. proposed an improvement to this interaction with the VIHapp system. It used attachments on the skis and poles, along with an armband to ease the communication between the guide and the skier through vibrations and motion sensing. However, it still required the need for a guide. The guide is an additional person who needs their own training

and is at risk of getting in the way of the skier's path. What if it was possible to build a system that removes the need for skiing guides altogether, while simultaneously allowing for the tracking of objects in sports in general?

This leaves a gap for the development of a more complex technical gadget to be used as a tool to help visually impaired people within the context of sports and PA. The system that will be developed and subsequently evaluated in this thesis aims to achieve this end through the means of an AI camera utilizing computer vision and machine learning for object detection. Developing such a system also presents an exciting opportunity to explore and analyze different methods for directional perception of objects; demonstrating and comparing their performance in this context.

1.2 Aim

The aim of this thesis project is to develop a physical prototypical platform that implements directional perception for the sonification of objects in an image captured by the AI camera, with the ultimate goal to increase the independence of blind people exercising sports. The project will also evaluate and compare two different object detection algorithms - one using the MobileNet SSD neural network included with the AI Camera [10], and the other making use of traditional color segmentation (ColSeg) methods. In the context of the project, their technical and practical performance will be analyzed and compared with each other. The project will be carried out in collaboration with a concurrent master student, who will develop the sonification of an object vector into spatial sound [11].

The goal of the final system is to provide fast, smooth performance in terms of directional perception, where all necessary calculations finish within the time it takes for a frame to be processed in order to avoid lag and uncomfortable sound glitches. The final prototype in terms of hardware is projected to be portable, compact, and light enough to be worn by an individual without causing any additional discomfort. It will consist of a Raspberry Pi, the AI Camera, a sound card, a pair of headphones, and a power source. An illustration of the projected final system can be seen in Figure 1.1.

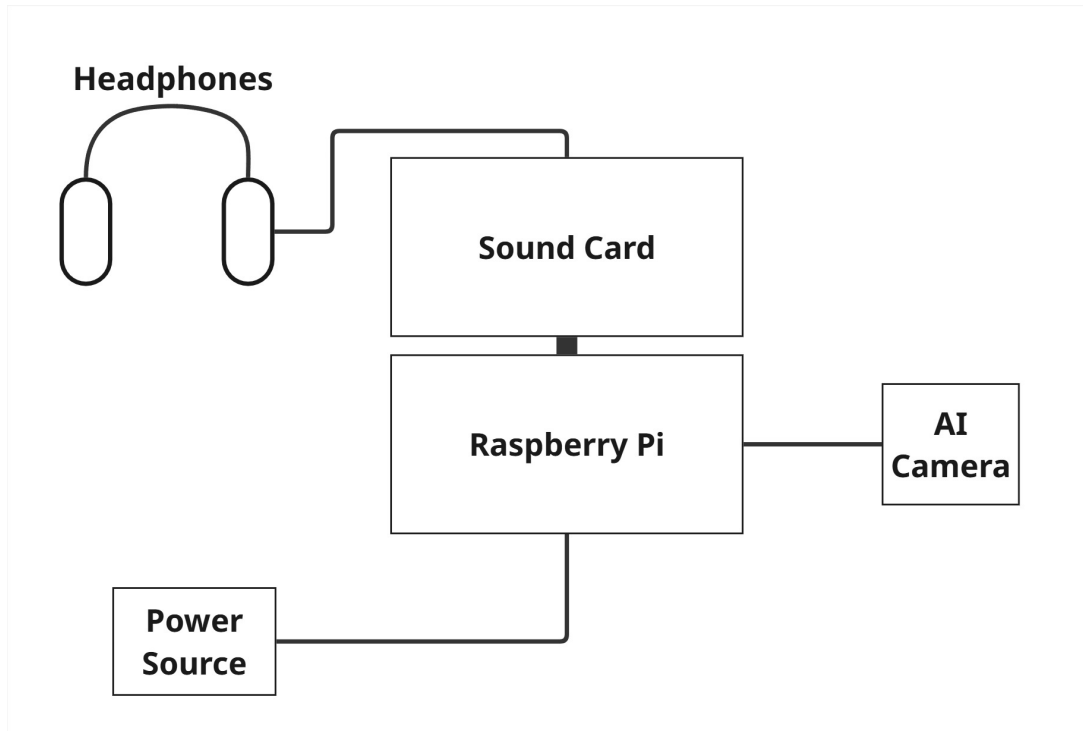


Figure 1.1: An illustration of what the final system is expected to look like. The power source will most likely be in the form of a power bank.

1.3 Research questions

1. How can an AI camera be utilized to provide accurate directional sensing for objects that are to be sonified?
2. What filtering strategies are required to parse the output from a bounding-box object detector to provide directional sensing, while meeting real-time system constraints?
3. How does the accuracy for directional perception and depth estimation of MobileNet SSD compare to ColSeg?
4. How does MobileNet SSD perform on a Raspberry Pi 5 compared to ColSeg within object sonification in terms of CPU load (%), thermal throttling limits ($^{\circ}\text{C}$), and sonification latency (ms)?

1.4 Delimitations

The object detection methods evaluated in this thesis are limited to traditional morphological color segmentation utilizing OpenCV functions, and the pre-trained MobileNet SSD model that is included with the AI Camera. Specifically the version 2 variant 320x320 of the model. No novel object detection techniques will be developed within the scope of this thesis.

The hardware used for the project is limited to what is provided by the department of electrical engineering at Linköping University.

A concurrent thesis is investigating algorithms for sonifying object points into spatial audio [11], therefore no sonification mapping will be studied here.

The experiments and tests conducted in the project are limited to indoor environments.



2 Theory and Hardware

The chapter explains the underlying theory within the framework of the project and gives a background to the hardware that was used.

2.1 Definitions

Throughout the report, a collection of terms are used which the reader is not expected to be familiar with beforehand. These terms are defined here.

ALSA (Advanced Linux Sound Architecture) - A software framework that allows for configuration of sound card device drivers [12].

Azimuth - Horizontal angular measurement relative to an origin.

Elevation - Vertical angular measurement relative to an origin.

HRIR (Head-Related Impulse Response) - An impulse response that is convolved with an input sound signal to induce spatial audio [13].

HRTF (Head-Related Transfer Function) - A mathematical principle that captures the dynamics of sound waves as they interact with the human body. It is used when implementing spatial audio [13].

HSV - A color format that defines a given color through the parameters hue (0-180), saturation (0-255), and value (0-255). Hue represents the color type, saturation is the colors vibrancy, and value denotes the brightness.

IIR (Infinite Impulse Response) - A filter consisting of a feedback loop, resulting in an infinite response when an impulse is added to the system [14].

multiprocessing - A package that allows for the implementation of parallelism in python [15].

Neural Network (NN) - A machine learning model that uses interconnected nodes to

mimic neurons. They are often used in vision projects [16].

Pipe - An object that allows for duplex flow of data between processes. Part of the multiprocessing package [15].

Python - A programming language [17].

Sonification - A way to represent data or information through the use of non-speech audio signals [18].

2.2 Hardware

The system is divided up into a number of hardware modules, which can be seen in Figure 2.1. The hardware consists of four main parts - a Raspberry Pi 5, a HIFI DAC HAT sound card, an AI Camera, and a pair of headphones.



Figure 2.1: The hardware constitution of the final system.

2.2.1 Raspberry Pi 5

The Raspberry Pi 5 is a small, single board computer. At the time of writing this thesis, version 5 is the latest and most powerful Raspberry Pi model in terms of CPU performance, RAM, and camera support [19]. These specifications are especially relevant for this project.

2.2.2 HIFI DAC Hat

The Innomaker DAC Hat is an external sound card that is made to enhance the sound output of the Raspberry Pi [20]. It connects directly on top of the Raspberry Pi 5 on the 40-pin connector and offers a simple setup that ensures quality sound playback. The sound card can be seen in Figure 2.2.

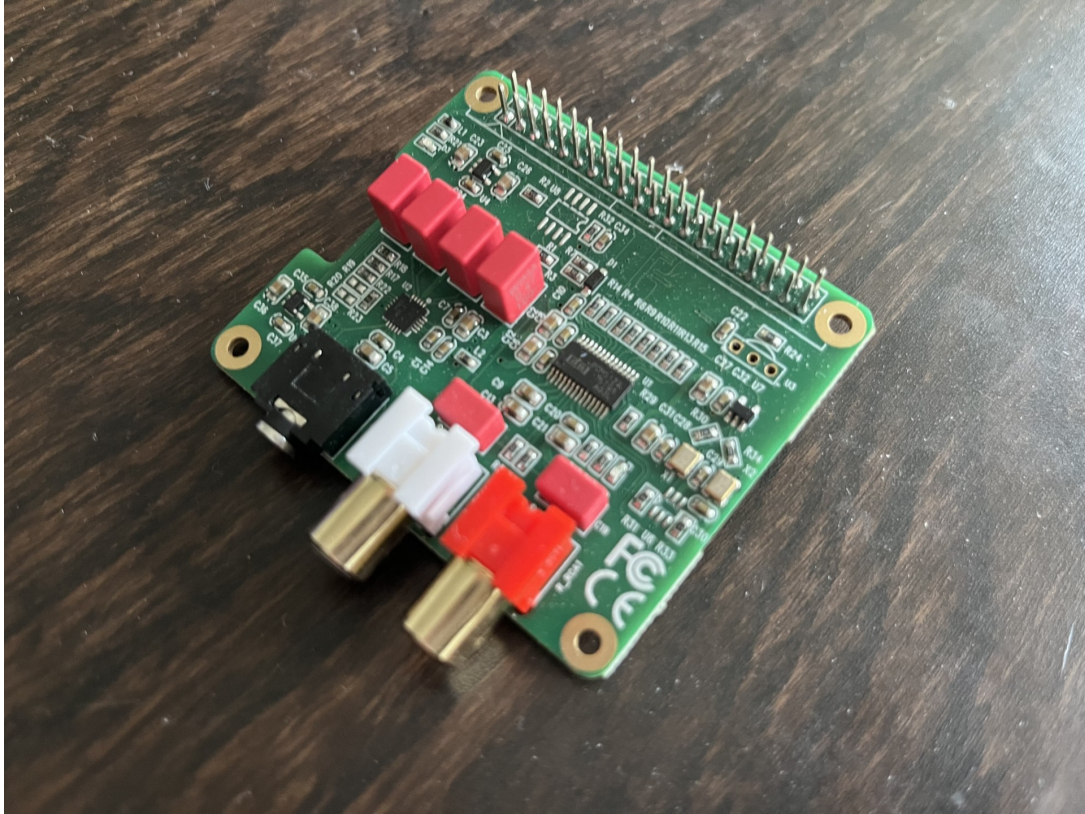


Figure 2.2: The HIFI DAC Hat sound card used in the final system.

2.2.3 AI Camera

The AI camera is a small and, at the time of writing this thesis, relatively new camera module created by the Raspberry Pi company [21], meaning it is compatible with the latest Raspberry Pi models. It utilizes the IMX500 vision sensor which can be loaded with a *NN* for use in computer vision projects. The IMX500 sensor handles the image data itself, storing relevant data from a given frame inside a tensor, providing a significant benefit by freeing up workload on the processor of the Raspberry Pi it is connected to [22].

The horizontal and vertical fields of view are specified to be $66.3 \pm 3^\circ$ and $52.3 \pm 3^\circ$ respectively. Since the camera is expected to act as a replacement for the users vision, this field of view is not enough, as the human eye has an approximate field of view of 150° in both horizontal and vertical directions, limited by cheeks, eyebrows, and the nose [23]. Of those 180° ,

the human eye has around 120° of depth perception to work with. To give an increased field of view, detachable lenses with wider angles were used in the project, see Figure 2.3

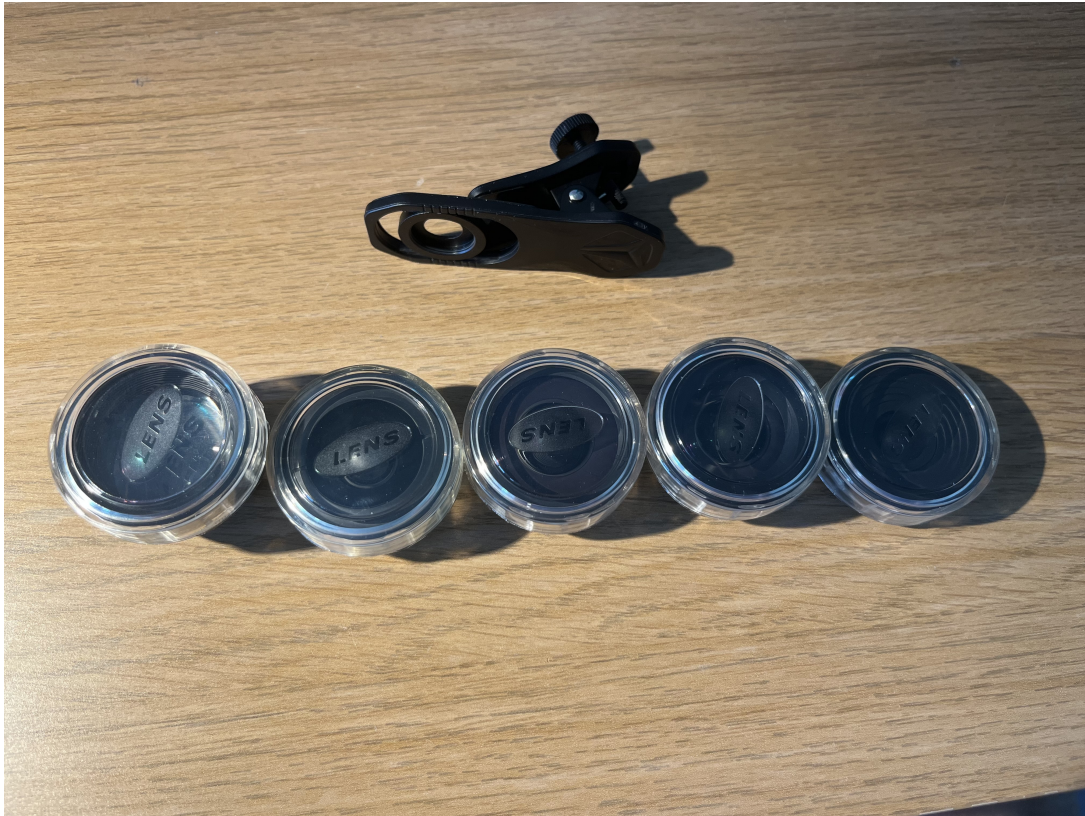


Figure 2.3: A collection of detachable lenses. From left to right: 2x telescopic, 195° fisheye, 170° wide angle, 110° wide angle, and a macro 10x lens.

2.3 Object detection methods

The two object detection methods used in this project are MobileNet SSD V2 320x320 and color segmentation.

2.3.1 MobileNet SSD V2 320x320

MobileNet SSD NN models make use of single-shot detection [24], hence the abbreviation. They do not require bounding-box proposals, making them much faster than other models, especially those of the Region-based Convolutional Neural Network (R-CNN) sort. Version 2, variant 320x320 is particularly fast compared to its previous iterations, since it scales down the captured image array to 320x320 pixels. This variant boasts “minimal decrease in mean average precision (mAP) of just 0.8 compared to its predecessor” according to Geethanjali, et al. The MobileNet SSD V2 320x320 comes pre-packaged with the Raspberry Pi AI Camera [10]. The metadata captured by the NN is stored in a simple tensor which includes data on the bounding-box, confidence values, and other useful information about the objects detected by the algorithm.

2.3.2 Color segmentation

Color segmentation is an image processing technique where certain objects and regions are isolated based on what pre-defined color ranges they correspond to [25]. For this project, the *HSV* color model will be used to define said color ranges.

2.4 Image projection

A camera has intrinsic and extrinsic parameters. The pinhole camera model offers a simple first-order approximation of these parameters [26], apt for the AI Camera with such a small lens and sensor. When applying the model, it is possible to describe the transformation and projection of a point from real-world coordinates onto the image plane with translational, t , and rotational, R , vectors. They are the cameras extrinsic parameters contained in the matrix E . The point is then mapped to image pixels through use of the camera matrix C , containing the cameras intrinsic parameters. This process can be described as

$$\bar{\mathbf{q}} = \mathbf{CE}\bar{\mathbf{q}}_w \Leftrightarrow \lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R_{11} & \dots & R_{n1} & t_{11} & \dots & t_{n1} \\ R_{12} & \dots & R_{n2} & t_{12} & \dots & t_{n2} \\ R_{13} & \dots & R_{n3} & t_{13} & \dots & t_{n3} \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}, \quad (2.1)$$

where λ is a scale factor, $\bar{\mathbf{q}}$ contains the coordinates of a point in the image plane, and $\bar{\mathbf{q}}_w$ contains the real-world coordinates. The parameters f_x, f_y are the cameras focal lengths and c_x, c_y its optical centers. The amount of rotational and translational vectors n is directly determined by the number of images used in the calibration training data set, see section 2.6.

2.5 Image Distortion

When a camera captures an image, there will inherently be some amount of distortion in the image caused by the lens [27]. Distortion can be expressed generally as

$$\bar{\mathbf{q}}' = g(\bar{\mathbf{q}}), \quad (2.2)$$

where $\bar{\mathbf{q}}$ are the ideal undistorted coordinates of a given point q in the image, and $\bar{\mathbf{q}}'$ are the distorted coordinates brought upon by the lens distortion function $g : \mathbb{R}^2 \rightarrow \mathbb{R}^2$. The two main types of distortion are those of the radial and tangential kind.

2.5.1 Radial distortion

Radial distortion and its prominence in an image depends on the field of view of the lens used when capturing said image. Wide angle lenses therefore have a greater degree of radial distortion compared to lenses with a smaller opening angle, leading to parts of the image appearing curved or warped [28]. This can be described mathematically by rewriting Equation 2.2 into

$$\bar{\mathbf{q}}'_r = g(\bar{\mathbf{q}}) = h(\|\bar{\mathbf{q}}\|) \cdot \bar{\mathbf{q}} = h(\|\bar{\mathbf{q}}\|) \begin{bmatrix} x \\ y \end{bmatrix}, \quad (2.3)$$

Where $\|\bar{\mathbf{q}}\| = r = \sqrt{x^2 + y^2}$ is the normalized Euclidean distance from the principal point $\begin{bmatrix} c_x \\ c_y \end{bmatrix}$ to a given image point $\begin{bmatrix} x \\ y \end{bmatrix}$. Hence, the image points are distorted in the radial directions in relation to the principal point, caused by the scaling radial distortion function $h : \mathbb{R} \rightarrow \mathbb{R}$. Radial distortion can either make the image appear shrunk (if $h(r) > 1$ for $r > 0$) or bloated

(if $h(r) < 1$ for $r > 0$), known as negative and positive distortion respectively, illustrated in Figure 2.4.



Figure 2.4: Examples of the two types of radial distortion. The leftmost image is undistorted, the middle image has negative (pincushion) distortion, while the rightmost image is affected by positive (barrel) distortion. From [27, p. 318], reproduced with permission.

The radial distortion function $h(r)$ can be approximated in a number of ways, e.g. through a Maclaurin series as

$$h(r) = a_0 + a_1r + a_2r^2 + a_3r^3 + \dots + a_6r^6 = 1 + a_1r + a_2r^2 + a_3r^3 + \dots + a_6r^6 \quad (2.4)$$

which can then be rewritten into

$$h(r) = 1 + k_1r^2 + k_2r^4 + k_3r^6, \quad (2.5)$$

where k_1, k_2 and k_3 are the distortion coefficients. The odd monomials are equal to zero since $h(r)$ must be an even function because of the rotational symmetry of radial distortion. The order of $h(r)$ changes depending on the severity of distortion in an image. For most cameras whose lenses are not exceptionally wide, a polynomial degree of 6 is often enough to model the distortion.

2.5.2 Tangential distortion

Tangential distortion, as the name implies, causes image points to be distorted in the tangential direction relative to the principal point. It is mainly caused by a physical misalignment of the lens and sensor [29] but can also be induced if a camera utilizes multiple lenses [27], the latter of which is not relevant in the case of the AI Camera.

Equation 2.2 can be rewritten into

$$\bar{\mathbf{q}}' = g(\bar{\mathbf{q}}) = \begin{bmatrix} x' \\ y' \end{bmatrix} = h(r) \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} 2p_1xy + p_2(r^2 + 2x^2) \\ p_1(r^2 + 2x^2) + 2p_2xy \end{bmatrix}, \quad (2.6)$$

where p_1 and p_2 are the distortion coefficients. The equation is a mathematical description of decentering distortion, a type of lens distortion that contains a tangential component along with an asymmetrical radial distortion [30]. From this, the tangential component can be isolated as such

$$\bar{\mathbf{q}}'_t = \begin{bmatrix} 2p_1xy + p_2(r^2 + 2x^2) \\ p_1(r^2 + 2x^2) + 2p_2xy \end{bmatrix} \quad (2.7)$$

The distortion coefficients are hence

$$d_c = (k_1, k_2, p_1, p_2, k_3) \quad (2.8)$$

2.6 Camera Calibration

One of the most important steps to consider when using a camera in the context of computer vision is that of calibration [31]. Capturing multiple images of a chessboard from different angles and using them as data sets for training and validation is a reliable way to achieve this end. In this project, the calibration compensates for the image distortion when mapping 2-dimensional image points to 3-dimensional coordinates, making it a necessary step in order to obtain accurate directional perception. Calibrating the camera also allows for the extraction of the cameras intrinsic and extrinsic parameters, described in section 2.4.

2.7 Coordinate mapping from a 2D pixel point to a 3D target vector

To create directional perception for a detected object, the vector to the objects center is calculated. The project makes use of a coordinate system that represents points as a target vector. It is expressed in 3-dimensions consisting of *azimuth* and *elevation* angles, which together make up the bearing, along with a distance to said point. A visual representation of the projects coordinate system can be seen in Figure 2.5.

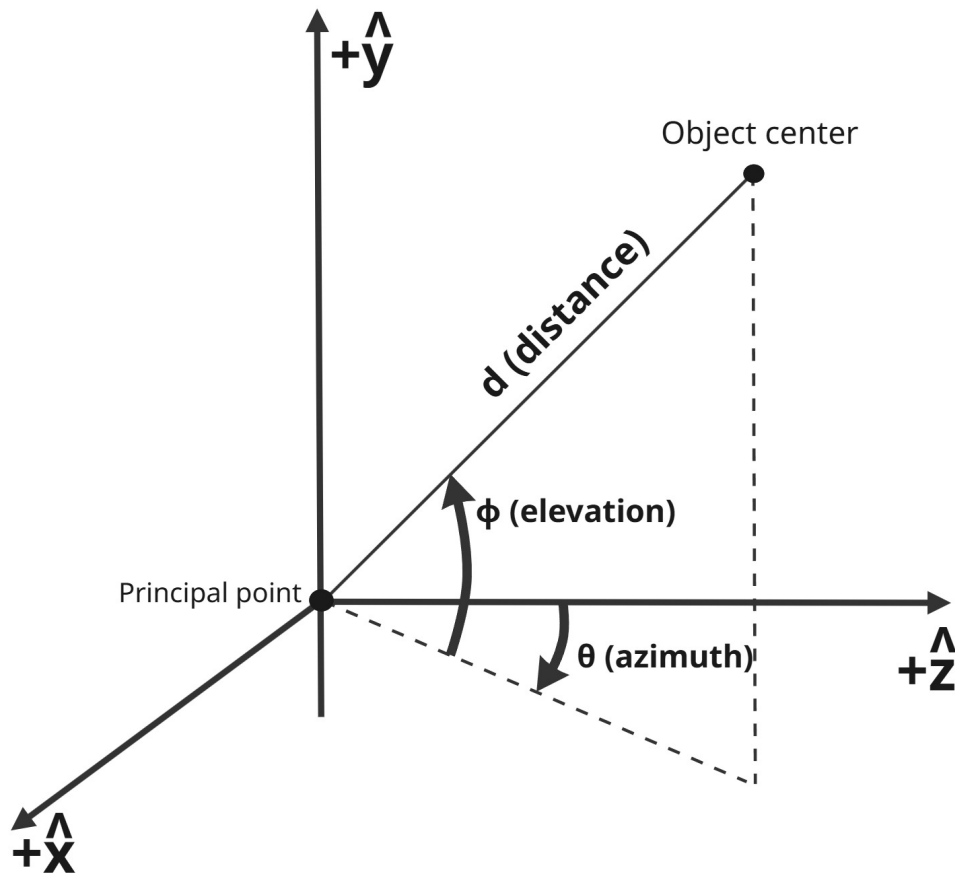


Figure 2.5: An illustration of how the coordinate system of the project is set up. The positive angular directions are up and to the right in relation to the z-axis.

The pinhole camera model describes the mapping of 3D world points onto a 2D image plane, the opposite of which also can be achieved. A pixel point $\bar{\mathbf{q}}$ in an image plane can be translated into the cameras internal coordinate system by inverting $\mathbf{C}$ in Equation 2.1, giving

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c = 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}^{-1} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad (2.9)$$

where x and y are the pixel coordinates. X_c , Y_c , and Z_c are the coordinates of the point expressed in the cameras coordinate system. Z_c remains unspecified since image points do not have a distance measurement. To calculate the azimuth and elevation, regular trigonometry is used in Equation 2.10 and 2.11 respectively.

$$\theta = \arctan\left(\frac{X_c}{Z_c}\right) \quad (2.10)$$

$$\phi = \arctan\left(\frac{Y_c}{Z_c}\right) \quad (2.11)$$

To calculate the distance, the focal lengths of the camera, pixel size of the detected object, and the real-world size of the detected object are utilized [32], giving the formula

$$d = \sqrt{\frac{w_r h_r \cdot f_x f_y}{w_p h_p}}, \quad (2.12)$$

where $w_{r/p}$ and $h_{r/p}$ are the real and pixel sizes respectively, and $f_x f_y$ are the focal lengths. The three coordinates are finally combined to form a point describing the target vector towards an object

$$\text{target vector} = (\theta, \phi, d) \quad (2.13)$$



3 Related Work

There has been a number of theses and projects over the years that broach subjects such as segmentation, object detection, directional perception, and tools for visually impaired people. The subjects said theses have touched upon are similar to the ones explored in this thesis, and will therefore be described in this chapter.

3.1 The SeeHear system

The idea of creating a tool for visually impaired people that maps visual events to sound queues is not new for this thesis. In 1987 [33], Nielsen et al. created an analog sonification platform that consisted of a lens, a small chip, and a pair of headphones. They used biological principles for sensory systems as a theoretical basis for their system. For the auditory perception, they formulated that the location of a certain sound source depends upon how a soundwave changes as it interacts with the human head and different parts of the ears, namely the pinna and tragus. Particularly, sound waves are delayed when interacting with the shape of the ear depending on the elevation of its source.

Visual perception in biology works through the lens of the eye projecting a visual scene onto the retinas photoreceptors, which together form a 2-dimensional array. The authors were able to express the eyes handling of visual scenes mathematically, in the sense that each photoreceptor generates “an analog neural potential proportional to the logarithm of the intensity at that point in the image”. The retinas output cells encode gradients and time derivatives of the intensity, allowing them to perceive motion signals, which in turn gives depth perception.

The thesis took these biological functions into account when constructing the final system, modeling it after them. The chip consisted of a 2D array of photosensors. It had delay lines next to every row of photosensors, creating a time delay for the sound which induced a vertical direction, while the sensors on the rows themselves gave the horizontal direction of a visual event. The chip computed the analog time derivatives of transient signals generated by light picked up by its array of photosensors, which also picked up light at different intensities from real-world visual events, giving a sense of where said visual events occurred. The eyes projection of a 3D world onto a 2D array in the retina was likened to the final systems use of the 2D array of photosensors. This, in turn, is similar to this projects mapping of 3D world coordinates onto a 2D image plane, see section 2.4.

The SeeHear project resulted in a system that achieved an impressive horizontal sonification of flashing stimuli, considering the analog nature of the tiny system and the low current levels it operated at. Unfortunately however, the experiments were plagued by a design defect. Incoming light affected the delay lines on the chip, since it operated on low currents and was very sensitive to stimuli as a result. This meant that the elevation of the sound could not be accurately represented by the chip.

Regardless of the design defect, the theory of mapping 3D world events and objects to an internal coordinate representation to then be used for sonification is essential for this thesis. The hardware constitution of the final system in and of itself acted as the main inspiration and starting benchmark for this project.

3.2 Morphological segmentation of sport scenes

In 2000 [34], Naemura et al. proposed a hierarchical structure for segmentation of high activity sports scenes consisting of two morphological techniques, Geodesic Binary Reconstruction (GBR) and watershed transformation (WST) based on markers. They did this with the goal of identifying athletes in sport scenes playing on television broadcasts. Since computers at the time were not able to differentiate between spectators and athletes without prior knowledge of the scene, the authors included an additional phase before the automatic segmentation. In this phase, an operator would define color histograms of the athletes' play area and their uniforms.

In the broadcast phase, the color histograms would first be thresholded, after which the GBR operation reconstructed images using geodesic distance, which is the distance between two points along a curved surface. This operation also defines markers to be used by the next step. Then, the marker-based WST operation would follow. It performed watershed transformation on the unclassified areas between the markers, effectively isolating the athletes and the playing area. In this way, the authors avoid oversegmentation by WST, a common problem, by watershedding only on smaller areas between the objects of interest.

The system was applied to soccer and baseball scenes, resulting in a significantly faster WST operation. For soccer scenes, the average computation time was 150 ms per frame, translating to 6.67 frames per second. While for baseball scenes, it landed at 120 ms per frame, i.e. 8.33 frames per second. For the time, the computational speed was fast in comparison to other contemporary methods, but in the context of this thesis, this is much too slow. The necessary calculations for directional perception using color segmentation preferably need to be finished within the time it takes to process a single frame.

The authors managed to prove that segmentation of image frames is a viable method for object detection, especially in live settings. The overarching ideas and intent of their system has been useful for the color segmentation implementation performed in this thesis.

3.3 You Only Look Once

Previous approaches to real-time object detection have also made use of fast NNs, such as in 2016 [35], when Redmon et al. developed the so-called You Only Look Once (YOLO) NN. Prior to the development and testing of this NN, most NNs used for object detection, such as R-CNNs, required bounding-box proposals and were therefore too slow to be used in real-time systems.

To solve this problem, the authors constructed YOLO so that it would be able to detect objects with one CNN using just a single evaluation of the image.

The project resulted in a regular YOLO and a fast YOLO version. Both managed to predict multiple bounding-boxes and the probabilities of their classes in one shot, while simultaneously reasoning an image globally, avoiding false positives, unlike R-CNNs. The regular YOLO system processed images at 45 frames per second, while the faster version reached

155 frames per second. However, the detection algorithm struggled with smaller objects, especially when they appeared in groups, e.g. flocks of birds.

The author's YOLO NN proves the efficacy and speed of single-shot NN models, a quality that it shares with the MobileNet SSD V2 320x320 used in this thesis. YOLO's shortcomings in relation to detecting smaller objects could also be relevant to this report, as it is entirely possible that MobileNet SSD suffers from the same problem.

3.4 MobileNetV2

Version 2 of MobileNet was first constructed in January 2018 and presented at the Computer Vision and Pattern Recognition (CVPR) conference in June of that same year [36]. It was developed by a team of engineers at Google Research and the resulting architecture marked a significant improvement in both accuracy and performance. For instance, it was shown to be 20 times more computationally efficient while using 10 times less parameters than YOLOV2 when tested in combination with their SSDLite object detector, benchmarked on the COCO dataset.

The authors took the original MobileNet architecture with its standard convolutions and factorized them into two layers; a depthwise convolution (one filter per input channel) and a 1×1 pointwise convolution. This reduced computational cost by a factor of 8 to 9 compared to standard convolutions, but also hampered the accuracy by a small amount. They also introduced linear bottleneck layers into the convolution blocks to prevent activation functions like ReLU from destroying information when collapsing a channel, due to the presence of non-linearities in the layers. Finally, they utilized so-called inverted residuals where shortcuts were created between the compressed bottlenecks instead of between the wide layers like in traditional residual block architectures. This improved performance by saving RAM space because the system no longer needed to store the wide layers in active memory, being able to store the compressed bottleneck layers instead.

While the development of YOLO in 2016 proves that single-shot detection is suitable for real-time applications [35], it is too computationally expensive for a device such as a Raspberry Pi 5. Therefore, the development of MobileNetV2 showcased by Sandler et al. bridges that gap and its findings are of direct interest to the system being developed in this thesis.



4 Method

The chapter describes the work process of the project. It outlines how the work was conducted in chronological order from beginning to end. It also describes the experiments performed; the results of which were useful for answering the research questions in chapter 1.3.

4.1 Hardware configuration

The hardware described in section 2.2 was the basis for the whole system, which means it had to be set up first. The DAC HAT and AI Camera were plugged into the Raspberry Pi while turned off and configured according to the instructions below.

4.1.1 AI Camera

The Raspberry Pi was booted up with the AI Camera connected to it. Terminal commands were used to update and upgrade the software on the Pi, as well as install the IMX500 firmware according to the instructions on the website [10]. This was done on a Pi with a freshly reset SD card. Lastly, the Raspberry Pi was rebooted.

4.1.2 HIFI DAC HAT

After plugging in the sound card, the Raspberry Pi was booted up and the config.txt file was modified according to the data sheet [20]. The code in Appendix A.2.4 was inserted in the *ALSA* configuration file `/etc/asound.conf` to ensure correct routing of the audio to the external hardware. Lastly, the Raspberry Pi was rebooted to apply the settings.

4.2 Calibration

The camera's video feed was turned on using the terminal command in Appendix A.1.1 to get a grasp of the quality of the camera. It was discovered that the camera was out of focus, after which the focus was manually adjusted using a circular tool included with the module, see Figure 4.1.



Figure 4.1: The tool used to manually adjust the focus of the camera.

The camera was calibrated by capturing 20 images of a chessboard with 14×10 squares in different orientations and from different angles using the camera's maximum resolution of 4056×3040 . Each image was captured separately with an added timeout of three seconds to allow for more accurate positioning of the camera. The terminal command used for this can be found in Appendix A.1.2, where the image number ranged from 0000 to 0019. The images were then divided up into datasets for training and validation.

The cv2 package [37] was used to find the corner points for each image of the chessboard, of which there were 13×9 , since the outer corners were ignored. If all corners were found for an image, the ground-truth 3D real-world points and the newly detected 2D image points were both saved in separate arrays. This was done for every image in the training set. Both of these arrays were used to compute the camera matrix, the translation and rotational vectors, and the distortion coefficients, see Equations 2.1 and 2.8. For this part, the template found on the OpenCV tutorial page for calibration was used [38], see Appendix A.2.1. The distortion parameter k_3 , see Equation 2.5, was removed using the flag **CALIB_FIX_K3** since it led to overfitting and consequent warping in the corners of any image it was applied to, see Figure 4.2. This gave the modified distortion coefficients in Equation 4.1. The camera matrix, along with the other results, were saved in a .npy file.

$$d_c = (k_1, k_2, p_1, p_2, 0) \quad (4.1)$$

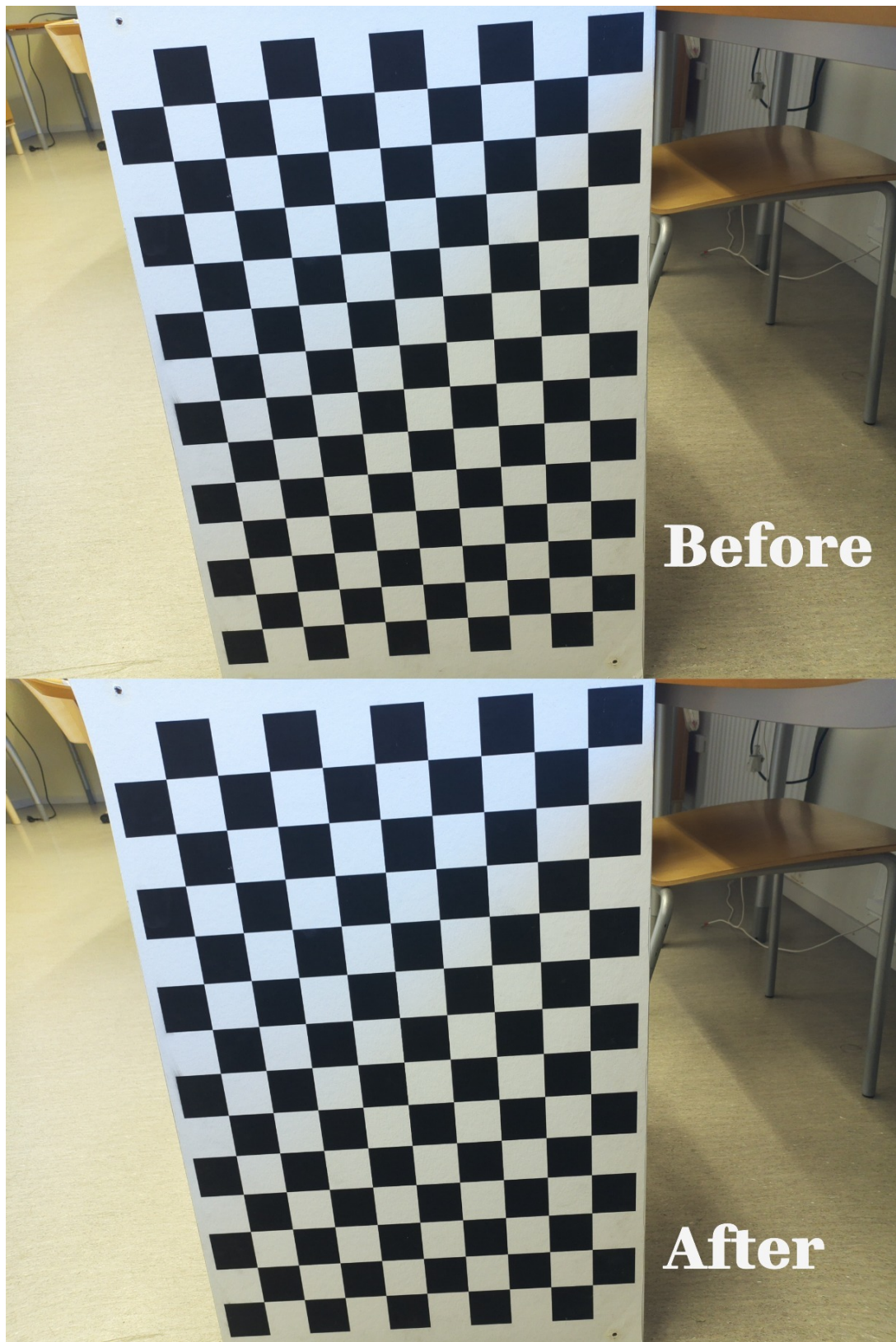


Figure 4.2: The chessboard before and after the overfitted calibration. A distortion in the corners of the after image can be observed.

4.3 Directional perception

The next step was to implement directional perception of the objects detected by the algorithms, as it was described in section 2.7. The data in the aforementioned .npy file was used when processing each frame of the camera image stream. For the purpose of directional perception, the calibration only needed to be run once because the resulting .npy file contained all the data necessary for the localization algorithms to work correctly.

The camera stream was started, followed by the processing of each frame. The data in the image frame was extracted, containing the coordinates and dimensions of the bounding-box of the detected objects. The pixel coordinates were converted to the cameras internal coordinates using the cameras intrinsic parameters from the calibration, before being mapped to angular coordinates. The angular coordinates represented the bearing of the middle point of the singular largest bounding-box. It was discovered that the cameras coordinate system had its origin in the upper left corner, and also defined the bounding-box origin in their upper left corner. To ensure that "up" was considered the positive elevation direction, Equation 2.11 was multiplied by -1 . Finally, the distance to the object was computed, creating a 3-dimensional vector to be used by the *HRTF* [11] in the sonification.

4.4 Sound playback

A number of different sounds were tried for use in the sound playback, whose functionality was delegated to a separate process so that it could run concurrently with the directional perception algorithms, not slowing down the loop that calculated the 3D vector. The target vector was shared between processes. The final sound that was chosen was a continuous humming. Persons using the system displayed the smoothest object tracking capability at this point. The process of finding the optimal sound is described in section 4.5.2.

4.4.1 HRTF calculation and frequency attenuation

The part of the implementation that made the humming sound play from the correct direction in the headphones and attenuate depending on the distance was provided by the concurrent master student who developed the sonification [11].

4.5 Experiments and tests

A number of experiments, measurements, and tests were performed on different parts of the system in order to evaluate its performance. The performance of MobileNet SSD and ColSeg was also compared. The results of these experiments is showcased in chapter 6.

4.5.1 Calibration

To ensure the validity and quality of the calibration, a number of measures for testing and validation were taken. The result of the calibration experiments can be seen in section 6.1.

Validation with native FOV

The training error, known as the re-projection error in the context of calibration, was computed. Each 3D real-world corner point was sent through the model created by the calibration process, which consisted of the camera matrix, distortion coefficients and the rotational and translational vectors. The output was the predicted 2D coordinates of each corner point, which together with the previously observed 2D corner points was used to compute the mean pixel training error per point. Both of these tests were performed using OpenCV templates [38], see Appendix A.2.2 and A.2.3.

To calculate the validation error, each image in the aforementioned validation set had to be processed. An OpenCV function was used to find the corner points in an image. If all the corner points were found, their coordinates together with the previously calculated camera matrix, distortion coefficients, and 3D real-world points were used to get the translational and rotational vectors of the validation image. These were then used as part of the model to predict the coordinates. The mean error between the real coordinates and the predicted ones was then computed, as with the re-projection error. This resulted in the mean pixel validation error per point.

The same operations for the training and validation errors were performed, but computed as a Root Mean Square Error (RMSE). Both the mean error and RMSE were recorded to gauge if and how outliers played a role in the calibration process.

Since the images in the training and validation data sets were shuffled randomly, the calibration and subsequent error calculations were performed five times to ensure a reliable trend in the values. Lastly, an image of a lined notebook was captured and undistorted using the camera matrix and distortion coefficients from the test case with the smallest validation RMSE. A lined notebook was chosen because it is generally easier to spot the effects of a calibration by how it affects straight lines. Gaussian blurring followed by adaptive thresholding was applied to both the original and undistorted image in order to isolate the notebook lines in each. A difference map between the original and undistorted image was then created to obtain a visual representation of how much, and in what way, the undistortion affected the image. The amount and percentage of line pixels that were shifted by the undistortion was also computed.

The five test cases and subsequent undistortion operation and difference mapping was performed for data-set splits of 60/40, 70/30, 80/20, and 90/10 in order to determine which split and accompanying smallest RMSE was the most appropriate to use for the directional perception algorithms.

Validation with lens attachment

Another method for validating the calibration was to test it on a wider lens. Investigating this was of interest because the AI Camera has a native field of view much smaller than that of a human eye, see section 2.2.3. A 110° wide lens attachment was used to give the camera a FOV more closely resembling that of the human eyes FOV for depth perception. A new data set of 20 images was captured with the wide lens attached. It was discovered that the lens attachment severely hampered the quality of any image taken, despite adjustments to the focus. This was caused by the lens attachment not being able to be properly fit over the camera, see Figure 4.3. This led to the algorithm detecting no corner points on certain images, meaning less corner points were detected overall. A new image was captured and undistorted using the data from the test case with the smallest validation RMSE, but no distance map was created here, as the effects of the calibration were much more noticeable. Just as with the default lens, all five tests were performed for data set splits of 60/40, 70/30, 80/20, and 90/10 in order to determine which split gave the most qualitative calibration result.

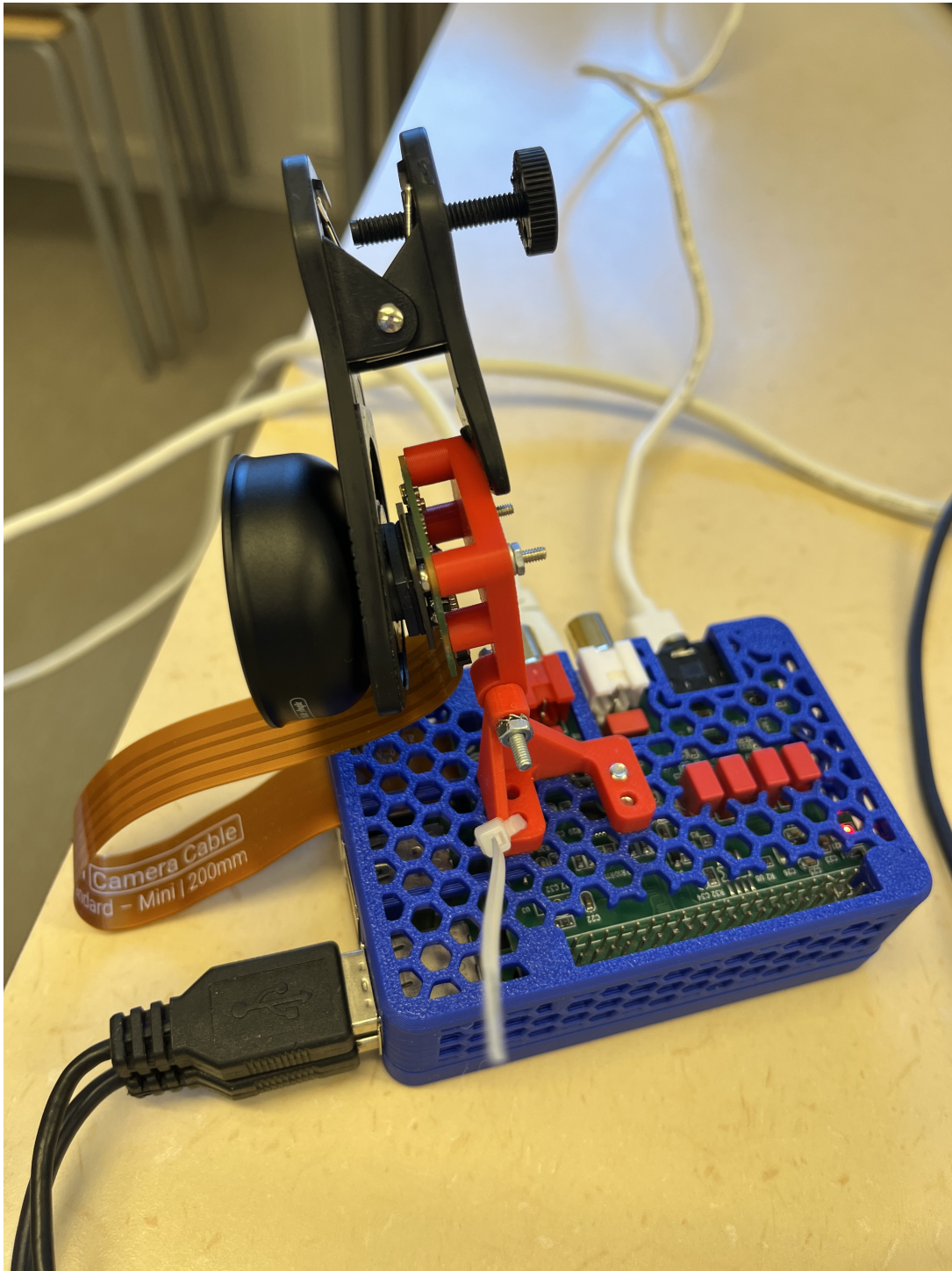


Figure 4.3: A 110° wide lens attachment fitted over the AI Camera. As can be observed, it does not entirely fit over the camera.

4.5.2 Directional perception

There were a number of both formative and summative experiments and measurements performed on the directional perception algorithms using MobileNet SSD and ColSeg. The goal

of the summative experiments was to test the algorithms performance and load on the Raspberry Pi system. The formative experiments were performed in order to test people's interaction with the system, if the directional perception algorithms could reliably induce an accurate object tracking capability by any given user, and if any changes to the system could be implemented based on user feedback. All of the tests were performed in indoor environments of $25m^2$ size with a room temperature of $23^{\circ}C$ that has either natural light or lamplight from the ceiling. The results of these tests are demonstrated in section 6.2.

Formative testing

Throughout the development of the project, formative tests were regularly conducted using both the author of this thesis, and other people not as familiar with the systems development, as subjects. Tests and observations were made in order to verify that certain parts of the system worked as expected, and where improvements could be made in regards to the directional perception for the user. The tests were conducted on healthy individuals with no visual impairment or blindness, rather the users would close their eyes and try to track an object using only the sound emanating from the headphones.

The most important part of the system that was continually tested and observed, especially at the start of development, was the directional perception. In the command line, the 3D target vector was printed every time a target vector was computed to see if the system correctly calculated the direction and depth of the object in frame. This was primarily how the revelation was made about the cameras coordinate system and bounding-boxes having their origin in their respective upper left corners.

One aspect of the system that required numerous formative tests was the choice of a sound signal. The initial version of the sound that played upon object detection was a finger snapping sound downloaded from an open source sound library. The sound file was perceived by users to be too long. It played much too infrequently and therefore made it hard for a user to accurately trace an object with their eyes closed. It was quickly switched out to a chirp signal, which decreased in length the closer the object was to the camera, mimicking a radar signal.

The chirp sound performed better with the persons trying it, but it came with its own problems. As an object got closer, the chirp eventually became unintelligible, instead being replaced by a clicking sound in the headphones at close distances. This was due to the signal being shortened too severely at close distances. It also was not easy for a person to trace an object at larger distances, because the same problem in terms of the slow playback frequency arose as with the finger snap.

The sound that received the best feedback from users was a continuous humming. When testing the system on different people, they were able to confidently follow the movement of the object with their eyes closed by pointing their finger at it. The attenuation of the frequency depending on distance was tested in a similar way, with the user reacting promptly and strongly as an object quickly approached the camera. These were performed with either a person or a balloon as the object.

Another part of the system that was tested was the ColSeg algorithm specifically. When testing it with a balloon, several persons reported that the sound stuttered from time to time, increasing in frequency the longer the system was active. This issue was completely absent when running the MobileNet SSD algorithm. To compensate, the resolution was scaled down by a factor of 4 to minimize the image array so it would not be as taxing on the hardware to perform the required image processing algorithms. This gave a better sound playback with less stutters, but was still not perfect.

Summative testing

When the development of the system was finished, a number of tests and measurements were performed to gauge the quality of the system and how the two directional perception

algorithms performed. The tests and measurements in list 4.5.2 were performed on both MobileNet SSD and ColSeg for three different cases of object detection. One where no object was detected, one with an object being present in the frame for the whole duration of the test, and one with an object entering the frame at approximately 30 seconds. The object used to test MobileNet SSD was a person, while the object used for ColSeg was a blue balloon. The reason for these choices of objects was that MobileNet SSD was incapable of detecting balloons. The closest resembling object was a sports ball, however, a balloon is able to move around and float on its own, slowly changing its position in the frame, making it more appropriate for both formative and summative testing. The objects were kept still for the duration of the tests.

- Measurement of Frames Per Second (FPS) inside the image frame handling loop every second for 60 seconds. This tests was performed to see if the FPS would vary or dip under different conditions and changes of the cameras observed environment.
- Measurement of the latency of the HRIR calculation inside the sound playback process alongside the FPS and amount of dropped target points every second for 120 seconds. This was to see if the HRIR could reliably be calculated within the time it took for the system to process a frame, and if not, how many target points were discarded as a result.
- The instantaneous CPU usage, system temperature, Resident Set Size (RSS), overall system RAM usage, FPS, HRIR latency, and dropped target amount were all measured alongside each other every second for 120 seconds. This was to see how much load the hardware was under during different conditions, and how it compared to the previous measurements conducted.

The CPU usage is the average percentage of the CPU in use across all of the Raspberry Pi cores. The temperature will show at what times the system heats up. Since the Raspberry Pi is known for thermal throttling, tracking the temperature potentially reveals bottlenecks and places where the Raspberry Pi might start under-clocking its processor as to not overheat. The RSS tells how many MB of RAM a specific Python script is using over time. This is a useful way to monitor memory leaks.

A number of stress tests were performed on MobileNet SSD and ColSeg, see list 4.5.2. These tests were of the longer variety, with random assortments of objects in different places in the frame, both moving and stationary. The camera itself was moved around from time to time. The measurements captured by these tests were of FPS, HRIR latency, dropped targets, CPU usage, system temperature, RSS, and overall system RAM usage.

- 120 second stress test where a balloon or person moved in, out, and around the frame in random, sporadic movements. This was to stress test the system on a moving object.
- 1 hour stress test where random objects such as a person, couch, chair, and bed where in the frame. For the ColSeg, it was to detect any blue segment in the frame with a minimum pixel area of 400. Blue was chosen because many different values and saturations of it were fairly common colors to find in the test environments. This was to test if the system could persist for a longer time without crashing, and if the audio playback would remain stable over time and for many different targets.
- 10 minute stress test of ColSeg with an increased minimum pixel area of 10,000. This was to test the ColSeg on larger, more relevant objects, to see if the performance differed from a pixel area of 400, since smaller color segments were discarded here.



5 System Description

The chapter starts with a brief overview of the resulting system after the work processes described in chapter 4 was finished. It then dives deeper into the configuration and structure of its software modules.

5.1 System overview

The software is made up of code written in *python*, divided into two main modules; one for handling the output from the camera, and the other for calculating the HRIR and playing the sound, described in sections 5.2.1 and 5.3 respectively. The *multiprocessing* package in Python is employed to allow for concurrent execution of both directional perception and sound playback. All of the code is located and has been written on the Raspberry Pi.

For the directional perception to work correctly, the camera matrix is needed, containing the focal length and optical center of the camera. The matrix is obtained by calibrating the camera using traditional computer vision methods, described in further detail in section 4.2.

A flowchart of the final system can be seen in Figure 5.1. At any time, the system can be easily terminated through use of the Ctrl-C keyboard interrupt.

5.2 Directional perception

The directional perception functionality handles the output from the respective object detection algorithms and maps said objects coordinates to a 3D vector. This 3D vector then gets sent to the audio process through a *pipe*.

5.2.1 Coordinate mapping

The camera matrix is extracted from the .npy file mentioned in chapter 4.2, along with the focal lengths contained within. The desired algorithm can be chosen with a command line argument; "nn" for MobileNet SSD or blank for ColSeg.

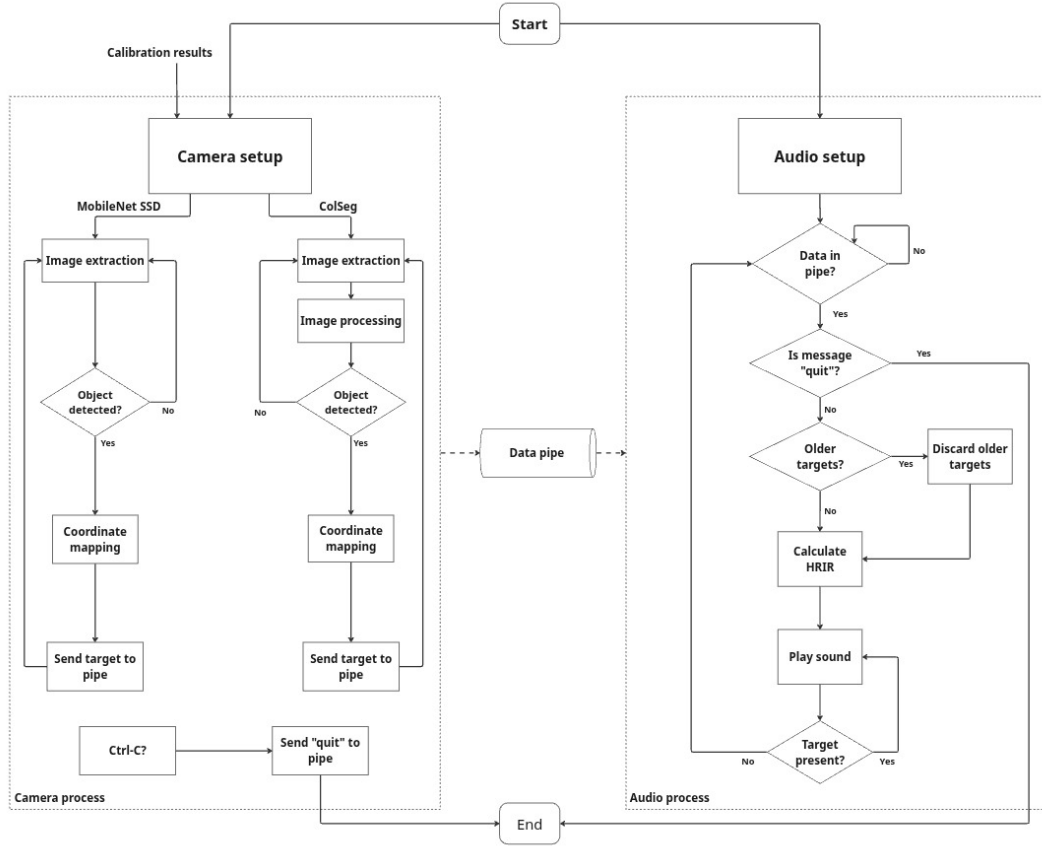


Figure 5.1: Flowchart of the final system.

Coordinate mapping with MobileNet SSD V2 320x320

The IMX500 sensor is initialized with the MobileNet SSD model, after which the camera is configured with a frame rate of 30 Hz and a resolution of 2028x1520 - half of the resolution at which the camera was calibrated. This is to ensure smooth performance and avoid overheating of the camera module. Because of the downscaled resolution, the camera matrix is also scaled down by half, see Equation 5.1. The camera starts, after which every frame is continuously processed in the same following way.

$$\begin{bmatrix} X_c \\ Y_c \\ 1 \end{bmatrix} = \left(\begin{bmatrix} 1/2 & 0 & 0 \\ 0 & 1/2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \right)^{-1} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (5.1)$$

The output tensor from the cameras metadata is extracted. It contains information about the bounding-box, confidence score, and category of each detected object. For all detected objects, only the one with the largest bounding-box (the nearest) with at least 50% confidence is processed. The pixel coordinates and dimensions of the bounding-box are used to find its middle point:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = s \begin{bmatrix} x_p + \frac{w_p}{2} \\ y_p + \frac{h_p}{2} \\ 1 \end{bmatrix}, \quad (5.2)$$

where x and y are the pixel coordinates of the middle point, x_p and y_p are the pixel coordinates of the bounding-box upper left point, and w_p, h_p are the bounding-box dimensions in pixels. The scale factor $s = 1$ in this case.

The middle points coordinates are translated to the cameras internal coordinate system using the camera matrix obtained through the calibration in chapter 4.2, see Equation 2.9.

Trigonometry is then applied to the objects coordinates to compute the azimuth and elevation angles, see Equations 2.10 and 5.3.

$$\phi = -\arctan\left(\frac{Y_c}{Z_c}\right) \quad (5.3)$$

The real distance to the object is computed using the focal lengths from the camera matrix in combination with the pixel dimensions and an average estimation of the real-world dimensions of said object. The dimensions correspond to width and height, see equation 2.12.

Since the output tensor expresses the object type as a floating point number from 0.0 to 89.0, the coco_labels.txt file from the picamera2 github repository [10] is utilized to look up the object name, for instance a laptop (72.0) or a person (0.0), with the float number representing the index of each row in the file. The real-world dimensions for all of the relevant objects are hard-coded into a .txt file for use in the distance calculation. If an irrelevant object, such as a plane or a boat, is detected, the dimensions are set to the default 1x1 m. This is to save time and to keep the project focused on objects that could reasonably be found in indoor environments.

After the distance is calculated, the three dimensional target vector (θ, ϕ, d) is sent through the pipe to be picked up by the audio handler.

Coordinate mapping with ColSeg

For this algorithm, the desired object and its dimensions need to be defined beforehand. Therefore, the localization should preferably take place in an environment where the goal object can be reliably represented by a single color range, such as a sports ball or team shirt. A color range is picked according to the HSV color space, along with a pre-determined real-life width and height for the object. The HSV color model is used here because of its ability to encapsulate colors that are more muted or of different brightness. The camera is configured with a frame rate of 30 Hz and a resolution of 507x380, scaling the resolution in section 5.2.1 down by a factor of 4 due to the problem described in section 4.5.2. All frames are then processed in the same way.

An array representation of the frame is extracted and converted from BGR to HSV color space. A mask of the image is created for the desired color range and its noise is cleaned up by performing an opening operation followed by a closing one. All contours corresponding to the chosen color that have a pixel area larger than N pixels are retrieved from the image. N can be changed depending on how small objects one wants to detect in any given use-case. A bounding-box for each contour is extracted, with the largest bounding-box being chosen for processing. The bounding-box middle point is calculated using Equation 5.2 with $s = 4$, scaling it back to get accurate coordinates. Then, the same arithmetic operations and coordinate mapping as described in section 5.2.1 are performed, excluding the calculation for the real-life area as it is already calculated beforehand.

5.3 Audio playback

The audio handler runs continuously in parallel to the directional perception and is triggered by the detection of data in the pipe. The target point is extracted and utilized for the computation of HRIRs for the left and right ear, which are then normalized to avoid spikes in the volume. The state of these two functions and the distance are updated every time a target point is found.

The sound-device module is used to create a stream of output data to the hardware, using a callback function as a real-time audio generator. The callback function creates a sawtooth wave to induce a humming sound which is attenuated as an object gets closer to the camera. Gapless playback is implemented by convolving the hum wave with the most recent HRIRs and then filtering the convolved signals with a real-time updating *IIR* filter. The two filtered signals are saved as a singular stereo signal to the output data.

A decorative element consisting of several thin, vertical black lines of varying heights, positioned to the left of the chapter title.

6 Results

The chapter presents the results of the projects summative tests described in section 4.5.2. This includes the final calibration results, and analyses of the performances and system loads for both MobileNet SSD and ColSeg on the Raspberry Pi 5 platform.

6.1 Calibration

The results and errors from the calibration are presented in this section, along with visual representations of the calibrations effects. All errors and measurements in this section are expressed in the unit of pixels.

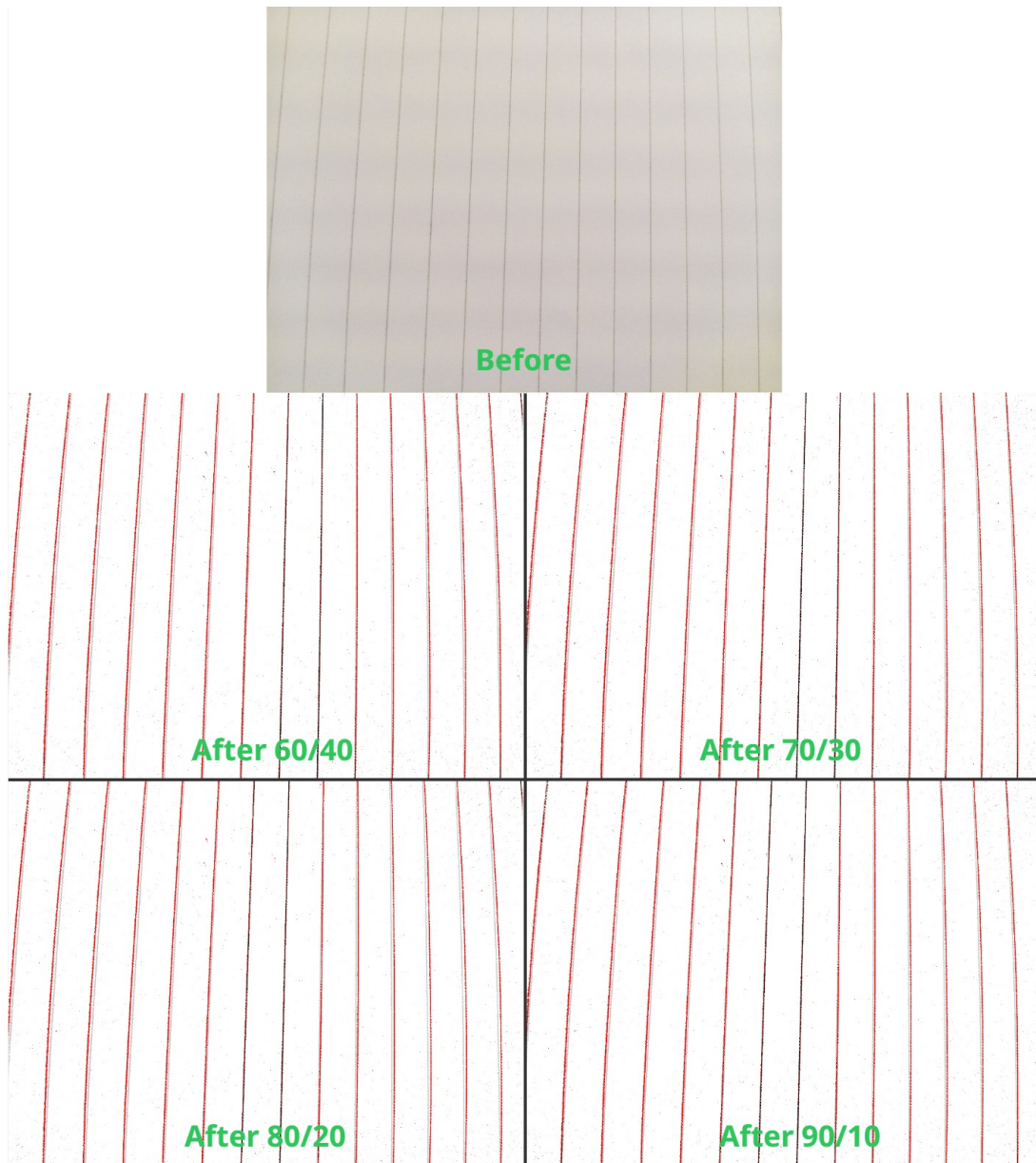


Figure 6.1: An image of notebook lines before the application of a calibration model, accompanied by difference maps of the image before and after the application of each calibration model using the best test case from each split. Each pair of lines consists of the line from the original image (gray) and the line from the image after a given calibration model was applied (red). Overlapping lines are black. A slight bulging of the lines away from the center can be observed, especially in lines further out to the edges of the image.

Table 6.1: Mean and sample standard deviation of the intrinsic camera matrix parameters across all data splits for default lens.

Parameter	Mean & Std
f_x	3076.691 ± 3.076
f_y	3074.115 ± 3.468
c_x	2030.274 ± 5.616
c_y	1521.458 ± 1.919

Table 6.2: Average calibration errors for each model.

Data split	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
60/40	0.094	0.115	1.139	1.348
70/30	0.102	0.100	1.234	1.144
80/20	0.102	0.099	1.240	1.096
90/10	0.096	0.130	1.141	1.523

Table 6.3: The extent to which each calibration model shifts the line pixels of the original image with the default lens.

Data split	Amount of line pixels shifted	Percentage of line pixels shifted
60/40	329,019	89.10%
70/30	322,206	87.25%
80/20	336,070	91.00%
90/10	319,710	86.57%



Figure 6.2: A new image of the chessboard before and after the calibration with the 110° lens using the best test case from each split.

Table 6.4: Mean and sample standard deviation of the intrinsic camera matrix parameters across all data splits for the 110° wide lens.

Parameter	Mean & Std
f_x	2043.100 ± 13.304
f_y	2044.914 ± 15.263
c_x	2093.006 ± 6.263
c_y	1382.187 ± 7.727

Table 6.5: Average calibration errors for each split with the 110° wide lens.

Data split	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
60/40	0.322	0.369	3.709	4.246
70/30	0.311	0.383	3.596	4.350
80/20	0.311	0.405	3.598	4.549
90/10	0.318	0.404	3.662	4.394

Table 6.2 displays the errors from the calibration performed with the default lens. The table shows the average of each error type across the five test cases for each data set split. Table 6.5 displays the same but for the calibration performed with the attached 110° wide lens. The result of all test cases for each split can be seen in Appendix A.3.1.

Figure 6.1 shows the effects of the calibration on an image of a blank page from a lined notebook. It shows the difference map between the original image and new image created by each calibration model. The calibrations effect on a new image of the chessboard with the wide angle lens can be seen in Figure 6.2. The calibration was performed for each split using the test case with the lowest RMSE.

The resulting camera matrices and distortion coefficients for each split using the default and wide lens can be seen in Appendices A.4 and A.5 respectively. All of the data sets have the distortion coefficients $p_1 \approx 0$ and $p_2 \approx 0$, meaning that no significant physical misalignment of the lens and sensor is present. The mean focal lengths and optical centers for the default lens have been computed in Table 6.1. The focal lengths are quite close to identical, confirming that the cameras pixels are square. The optical centers lie around 2028×1520 , which are the pixel coordinates of the principal point. This indicates a successful calibration.

The intrinsic camera parameters for the wide lens can be seen in Table 6.4. The focal lengths are shorter with the lens attachment, which makes sense since a wider FOV yields a shorter focal length. The optical centers are considerably further away from the principal point and both parameters have greater standard deviations. This is likely due to the misalignment of the lens attachment, and hampered quality of the images in the data set, as fewer corner points are detected as a result.

6.1.1 Determining the optimal calibration model for directional perception

Choosing the optimal calibration model for use in the directional perception is paramount, as the accuracy of the coordinate mapping depends on it. To determine the optimal model, both the error values and the calibrations performance (in terms of undistortion) on a new image have to be taken into account. Looking at the errors in Table 6.2, it is tempting to choose the model with the 80/20 split in data, as it has the lowest mean error and, more crucially, the lowest RMSE for the validation data set. However, when looking at the effects of the undistortion in Figure 6.1 on an image of notebook lines, it can be observed that the calibration does not compensate for the distortion, which is seen most clearly in the left-most and right-most lines, as their bend is not corrected in any of the "after" images. On the contrary, the calibration seems to give the image a small amount of radial distortion in the form of lines that are bulging out, known as barrel distortion. The goal of the calibration is to compensate for distortion, not cause it. Therefore, this result likely points to overfitting, even if it is by a small degree and barely visible to the naked eye. The large gap between the Mean Error and RMSE values indicates the presence of large outliers, which are caused by overfitting. Additionally, Table 6.3 shows that the 80/20 model shifts the lines in the image the most out of all the models, meaning 80/20 is the model causing the most distortion out of all of them.

Because the 80/20 model has such a low average validation RMSE but bends straight lines the most out of all the models, it means that the model is accurate where the chessboard is concerned, but fails to generalize for the lenses characteristics as a whole. Therefore, either

the 70/30 or the 90/10 data split should be chosen, as they both shift the least amount of pixels. In this case, the 70/30 model is more reliable, as its errors are more consistent between test cases compared to the 90/10 model, see Appendix A.3.1. It also has a relatively low validation RMSE, striking a favorable balance. Therefore, the resulting intrinsic parameters of the calibration with the 70/30 model was used for all subsequent experiments in section 6.2.

6.1.2 Determining the optimal calibration model for the wide lens

The 110° wide lens would not be used for directional perception, as it hampered the quality of the image and could not be fitted with sufficient stability over the camera because of its weight and size. Its weight compared to the AI Camera could also create a risk of damaging the hardware as the camera moved around during the directional perception experiments.

Choosing the optimal calibration model here was a significantly simpler matter. The lowest RMSEs belonged to the 60/40 and 70/30 splits, and the best calibration result on an image was achieved by the 70/30 model. As seen in Figure 6.2, the 60/40 model still had some left-over barrel distortion, while the later models started introducing a pincushion distortion. The 70/30 model is therefore the optimal one in this case.

6.2 Directional perception

The section presents the results of tests on MobileNet SSD and ColSeg for performance, bottlenecks, system loads, and stress for different cases.

6.2.1 Controlled tests

The following tests were conducted under controlled conditions where only one object was detected at a time. The first type of test was one with no object present, to get a baseline for the performance. The second was with an object present throughout the entire duration of the test, and the last was with an object entering the frame at approximately 30 seconds. These tests were conducted in order to isolate the effects different kinds of inputs would have on the metrics of the hardware and software.

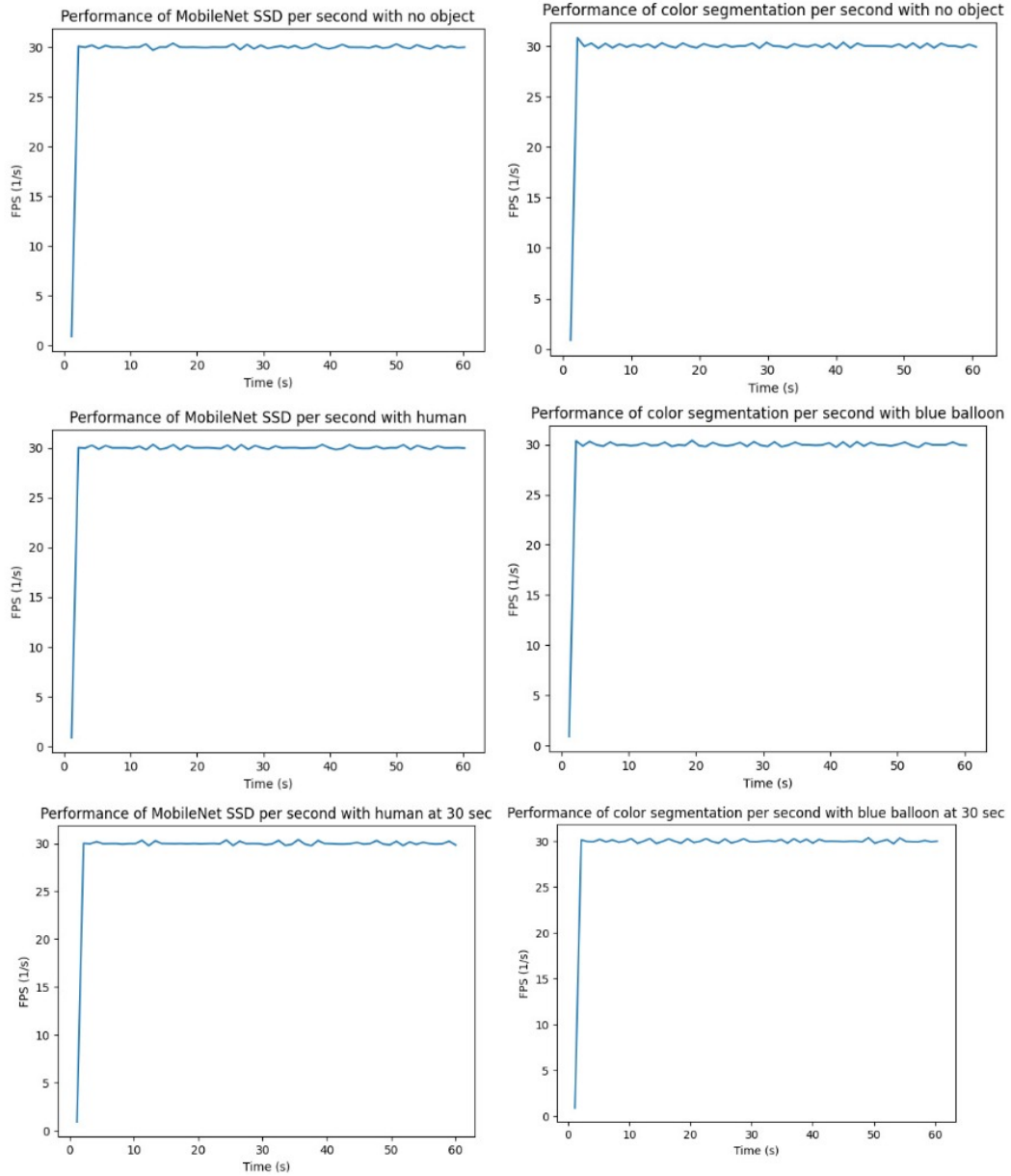


Figure 6.3: FPS count for 60 seconds using MobileNet SSD (left) and ColSeg (right) for different directional perception cases. For the third case, the object is detected by the algorithms at 30.89 seconds and 31.33 seconds respectively.

Table 6.6: Mean FPS counts for different object detection cases for 60 seconds.

Case	Algorithm	FPS
No object	MobileNet	29.52
Human present	MobileNet	29.52
Human present from 30.89s	MobileNet	29.52
No object	ColSeg	29.54
Balloon present	ColSeg	29.53
Balloon present from 31.33s	ColSeg	29.52

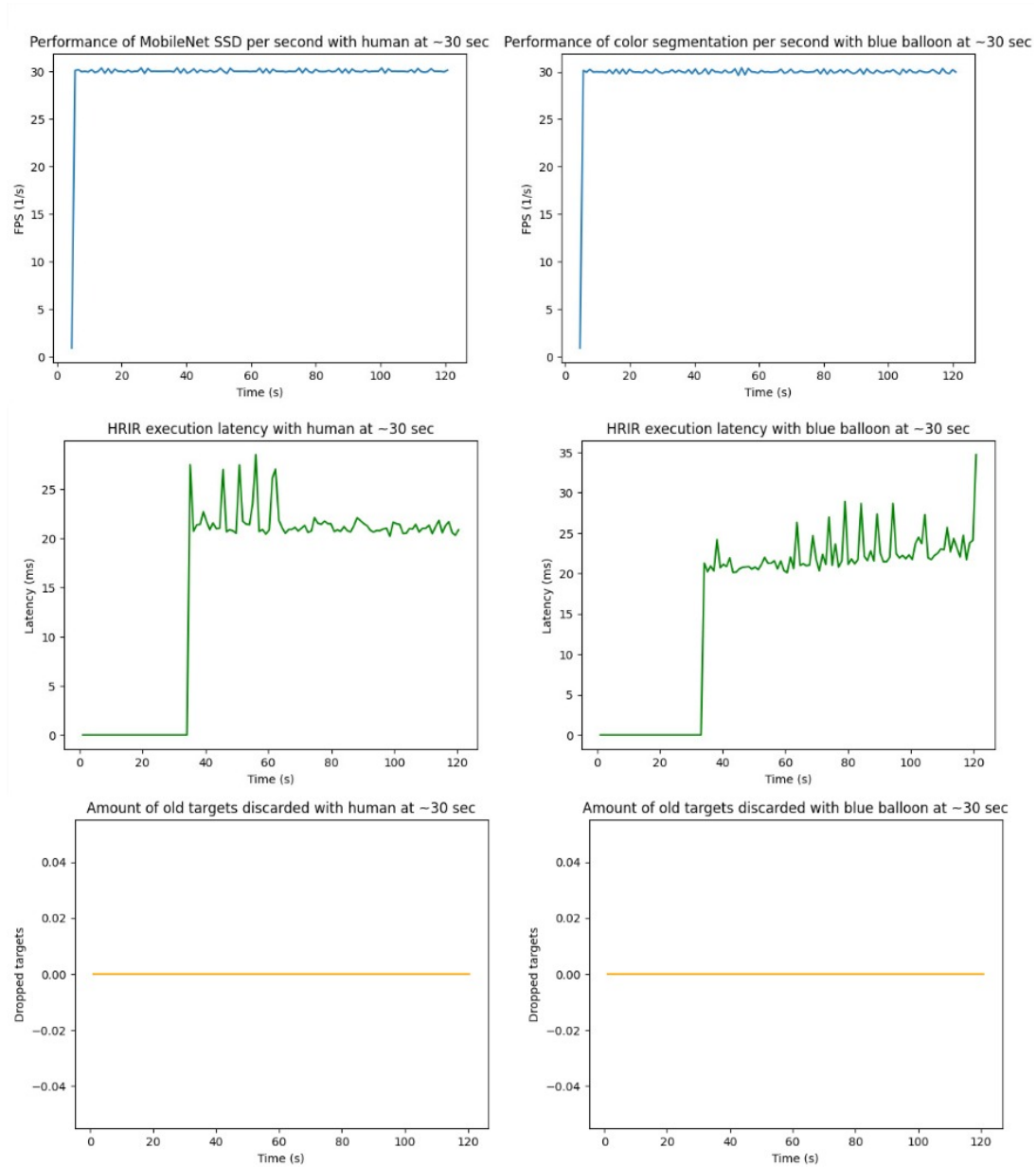


Figure 6.4: FPS count and HRIR latency over time using MobileNet SSD (left) and ColSeg (right) for when an object enters the frame at approximately 30 seconds. The object is detected by the algorithms at 34.71 seconds and 33.17 seconds respectively.

Table 6.7: Mean FPS counts and HRIR latencies for an object detected at approximately 30 seconds onward for 120 seconds.

Algorithm	FPS	Frame duration (ms)	HRIR latency (ms)	Detection timestamp (s)
MobileNet	30.02	33.31	21.55	34.71
ColSeg	30.02	33.31	22.20	33.17

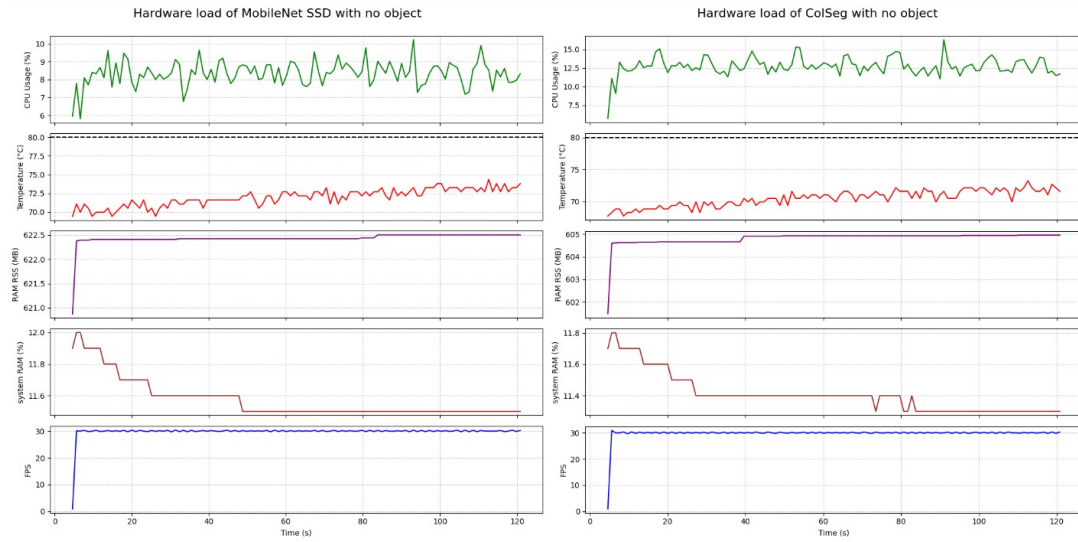


Figure 6.5: System load metrics using MobileNet SSD (left) and ColSeg (right) when no object is present for 120 seconds. The graphs show the following metrics from top to bottom: CPU usage (green), Temperature (red), RSS (purple), RAM (brown), and FPS (blue). The 80° C limit is marked with a dotted line.

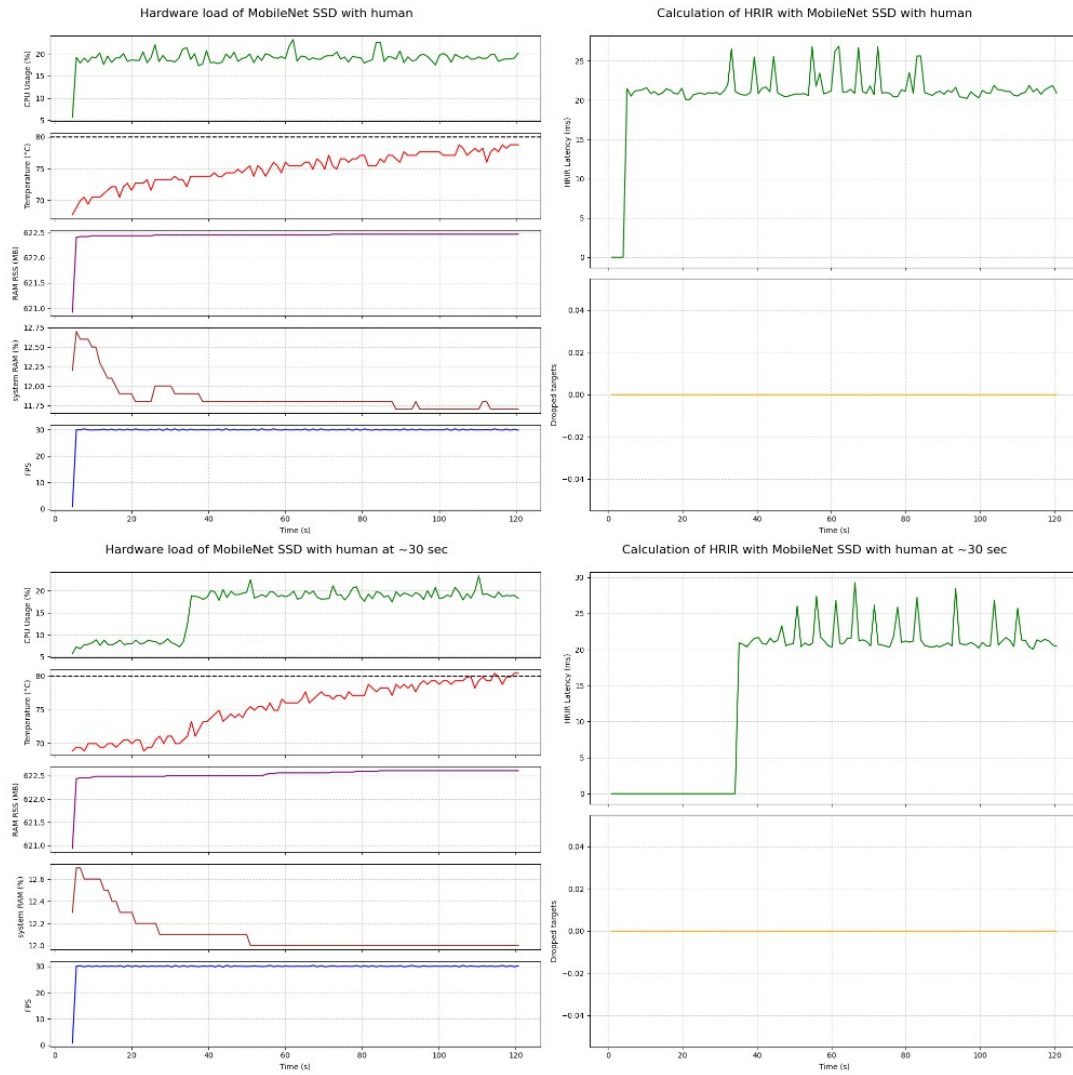


Figure 6.6: System load metrics next to HRIR latency using MobileNet SSD for two cases of human detection for 120 seconds. The graphs on the left hand side show the following metrics from top to bottom: CPU usage (green), Temperature (red), RSS (purple), RAM (brown), and FPS (blue). The graphs on the right hand side show the latency and dropped targets inside the audio process. The 80° C limit is marked with a dotted line.

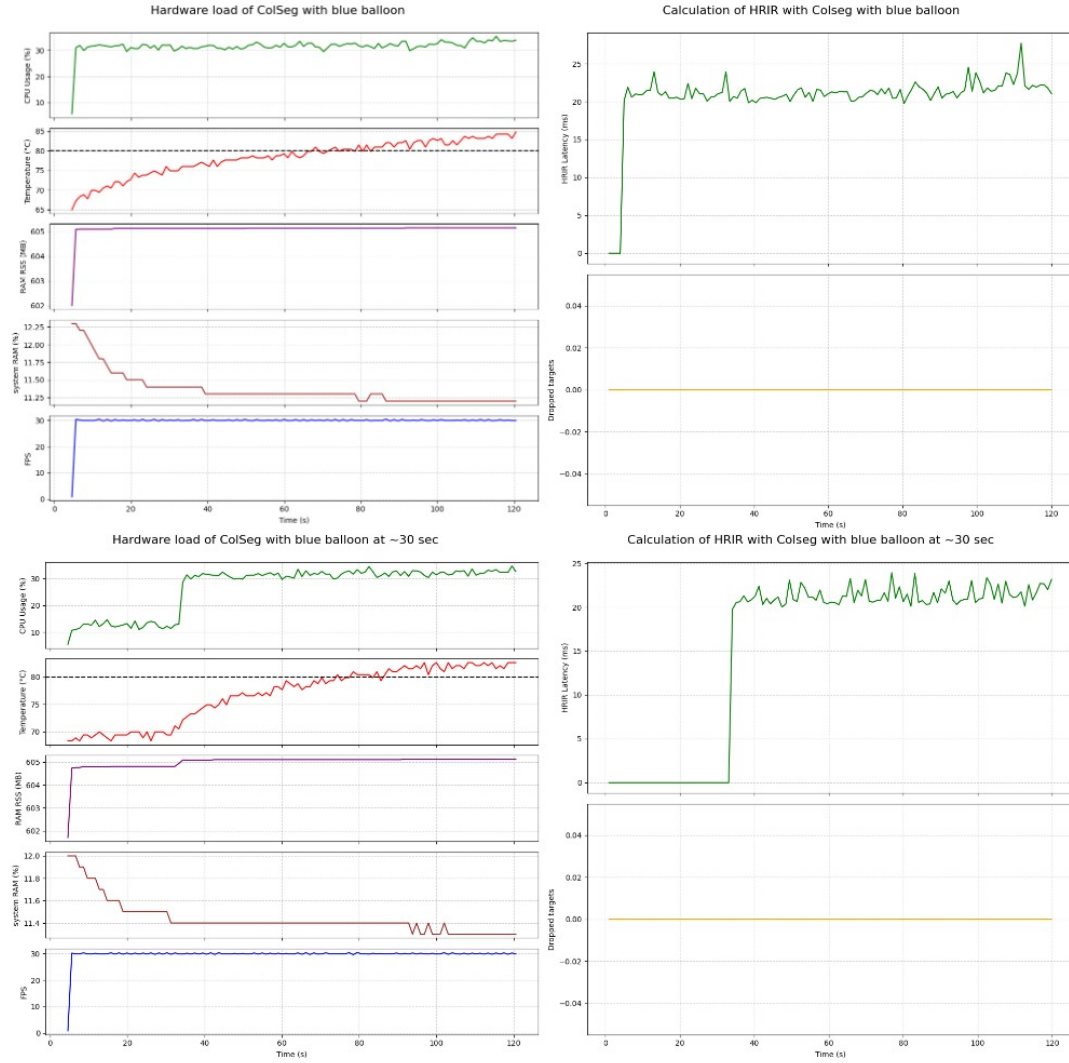


Figure 6.7: System load metrics next to HRIR latency using ColSeg for two cases of blue balloon detection for 120 seconds. The graphs on the left hand side show the following metrics from top to bottom: CPU usage (%), Temperature ($^{\circ}\text{C}$), RSS (MB), RAM (%), and FPS. The graphs on the right hand side show the latency and dropped targets inside the audio process. The 80°C limit is marked with a dotted line.

Table 6.8: Mean system load metrics for 120 seconds.

Case	Algorithm	CPU usage (%)	Temperature ($^{\circ}\text{C}$)	RSS (MB)	RAM (%)	FPS	Frame duration (ms)	Latency (ms)
No human	MobileNet	8.35	71.93	622.43	11.57	29.78	33.58	N/A
Human	MobileNet	19.04	75.03	622.44	11.85	29.78	33.58	20.76
Human from 34.25s	MobileNet	19.13	77.29	622.57	12.02	30.04	33.29	21.38
No balloon	ColSeg	12.73	70.51	604.82	11.41	29.79	33.57	N/A
Blue balloon	ColSeg	31.63	78.24	605.11	11.36	29.78	33.58	20.579
Blue balloon from 33.46s	ColSeg	31.79	79.50	605.11	11.37	30.04	33.29	21.09

Measurements of FPS and HRIR bottleneck

The FPS tests showcased in Figure 6.3 and its data in Table 6.6, shows that the directional perception process has a baseline stable FPS regardless of input and algorithm chosen for 60 seconds.

The HRIR latency and FPS tests showcased in Figure 6.4 have their data recorded in Table 6.7. They show that the HRIR calculation inside the audio process consistently manages to finish within the duration of a frame for the duration of 120 seconds, as no old targets from previous frames are dropped to accommodate for new ones.

Measurements of system load

Figures 6.5, 6.6, and 6.7, and their adjacent Table 6.8 continue to show the systems stability. MobileNet SSD and ColSeg put similarly minimal loads on the system regardless of input. The RSS and overall RAM shows that there are no memory leaks for either of the algorithms, which indicates a stable code and software. The overall system RAM usage remains minimal and stable over time, hovering at 11-12%. Finally, the FPS and HRIR remain stable just the same, hovering around 30 FPS and 21 ms respectively.

However, a few key differences are noted even in this small time span. MobileNet uses less of the CPU on average, spanning approximately 8-19% compared to ColSeg which uses around 12-31%. The temperature remains similar for both, at $72^{\circ} - 77^{\circ}\text{C}$ and $71^{\circ} - 80^{\circ}\text{C}$ respectively, but a potential red flag is that ColSeg manages to confidently overshoot the 80°C line within its first 120 seconds.

6.2.2 Stress tests

The following tests were conducted under more spontaneous conditions where a larger variety of objects could potentially be detected at any time. The objects in these tests were moved in, out, and around the frame in random sporadic movements. The camera itself was also moved to focus on different parts of the test rooms from time to time. These tests were conducted in order to evaluate the systems performance under stress and more unpredictable circumstances, as well as for longer times, more closely resembling sport scenes. The 1 hour long stress tests in particular, were conducted in order to gauge whether the system could survive under load for an extended period of time, such as a hypothetical sports game. The system needs to be stable during this entire duration to prove that it can be used for directional perception for visually impaired individuals within the context of PA and sports.

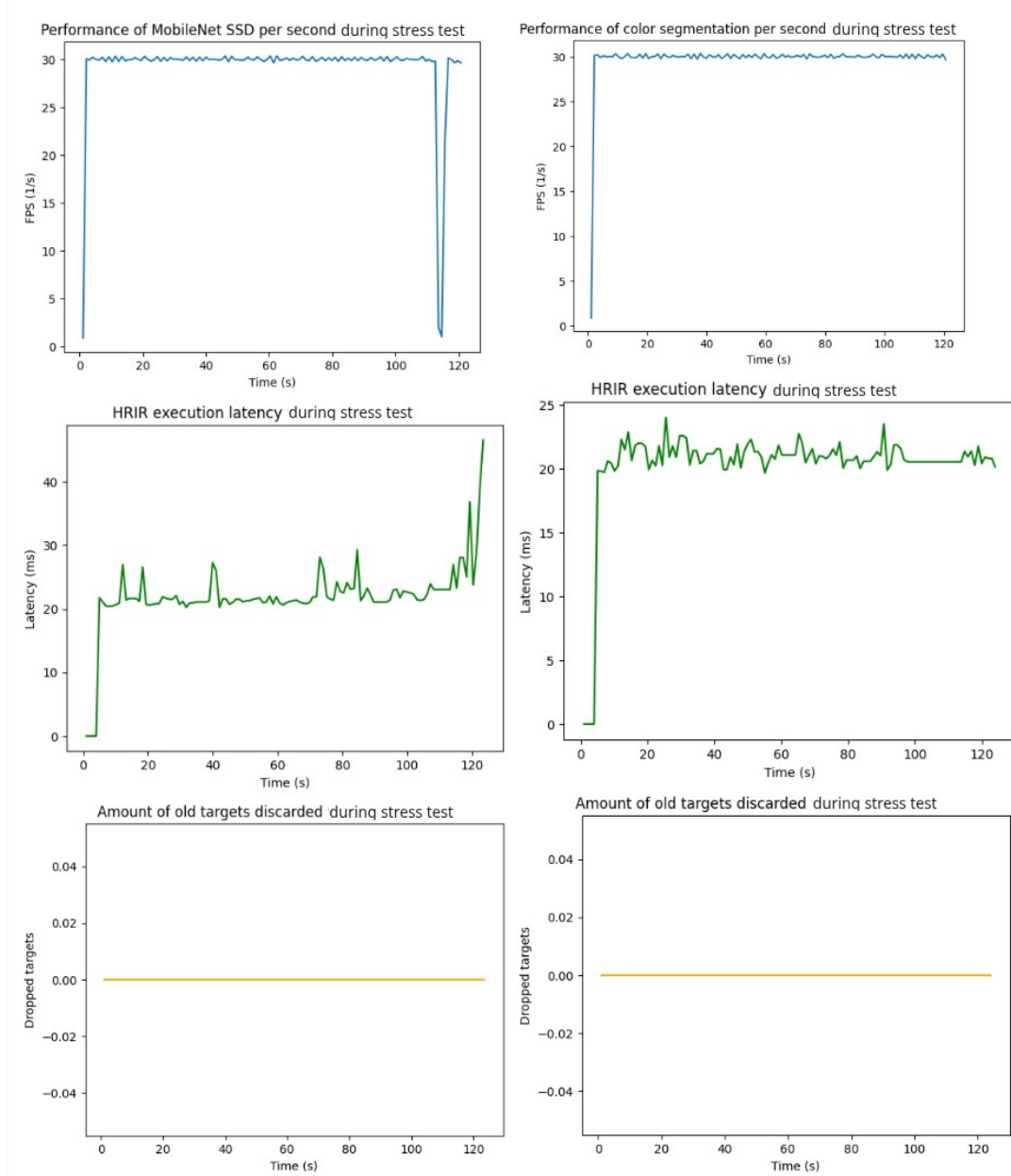


Figure 6.8: Stress test with a human (left) and blue balloon (right) moving in, around, and out of frame randomly for 120 seconds. Bottleneck for MobileNet SSD occurs at index 115.

Table 6.9: Mean FPS counts and HRIR latencies for 120 second stress test.

Algorithm	FPS	Frame duration (ms)	HRIR latency (ms)
MobileNet	29.20	34.25	21.97
ColSeg	29.77	33.59	20.30

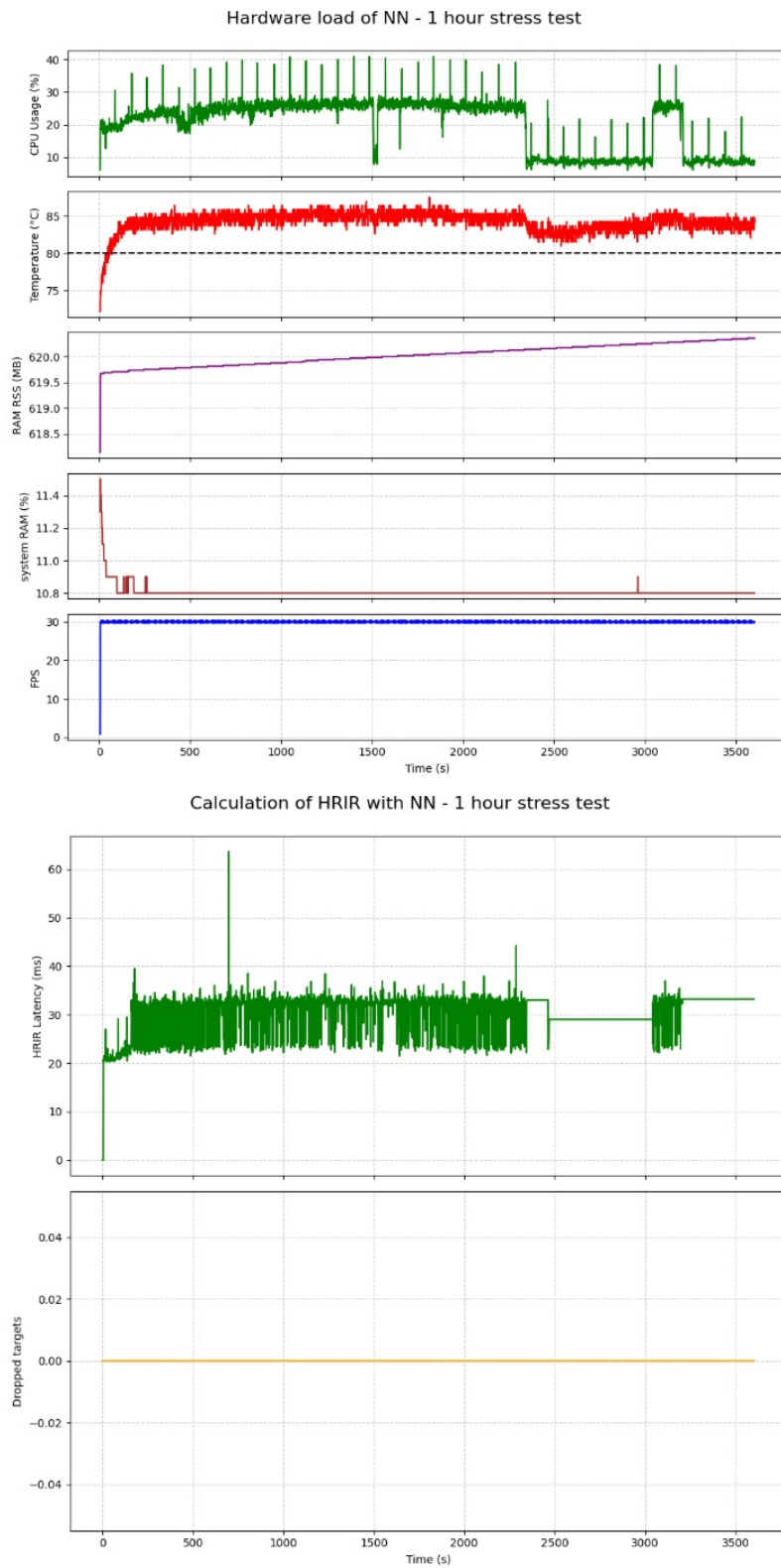


Figure 6.9: System load metrics and HRIR latency using MobileNet SSD for 1 hour. The graphs show the following metrics from top to bottom: CPU usage (%), Temperature ($^{\circ}\text{C}$), RSS (MB), RAM (%), FPS, HRIR latency, and dropped targets.

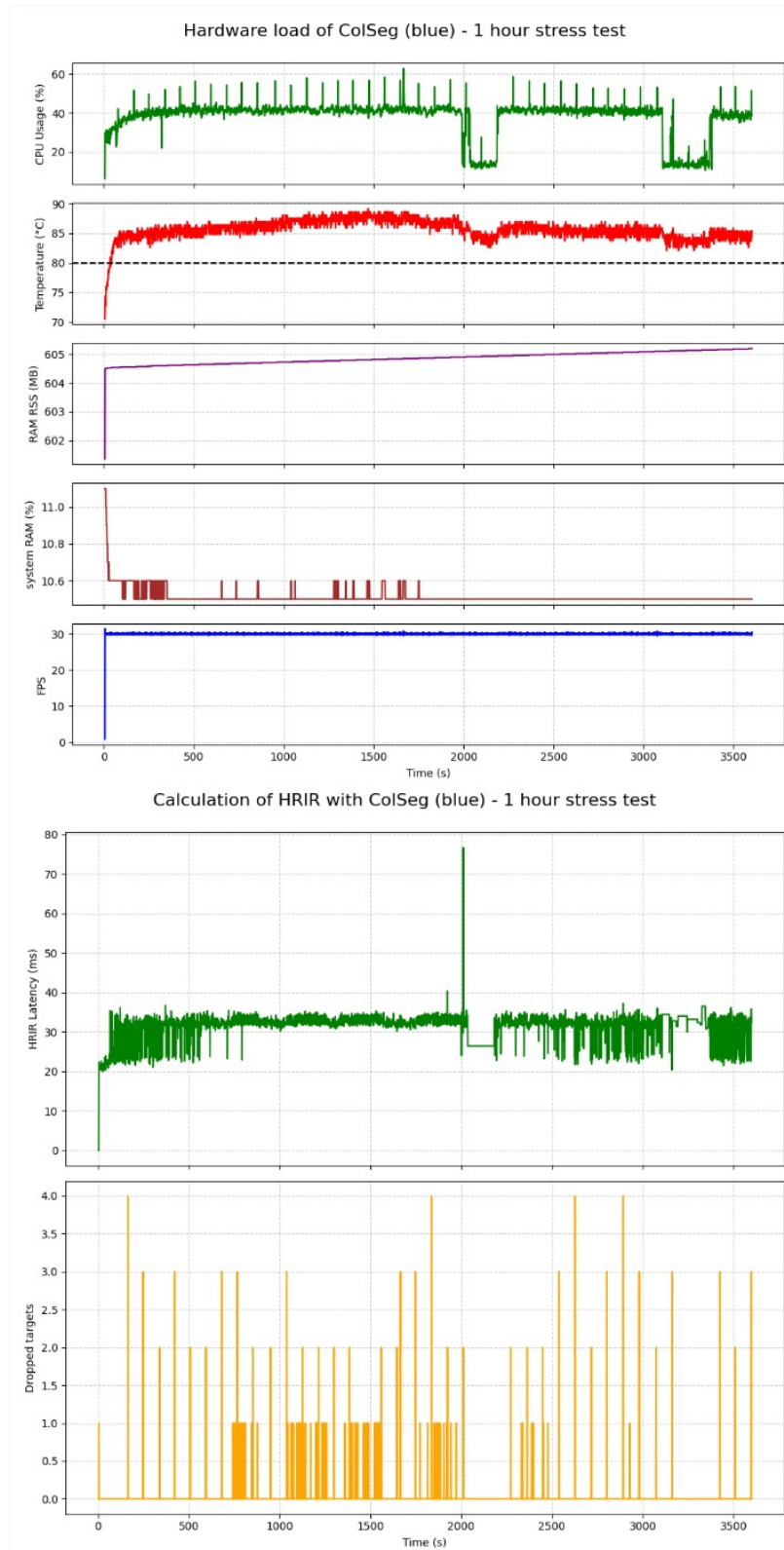


Figure 6.10: System load metrics and HRIR latency using ColSeg for 1 hour. The graphs show the following metrics from top to bottom: CPU usage (%), Temperature ($^{\circ}\text{C}$), RSS (MB), RAM (%), FPS, HRIR latency, and dropped targets.

Table 6.10: Mean system load metrics from 1 hour stress test.

Algorithm	CPU usage (%)	Temperature (°C)	RSS (MB)	RAM (%)	FPS	Frame duration (ms)	Latency (ms)	Dropped targets (per min)	Dropped targets (total)
MobileNet	20.06	84.31	620.03	10.81	30.03	33.30	30.28	0	0
ColSeg	37.65	85.68	604.86	10.51	30.04	33.29	31.60	3.05	180

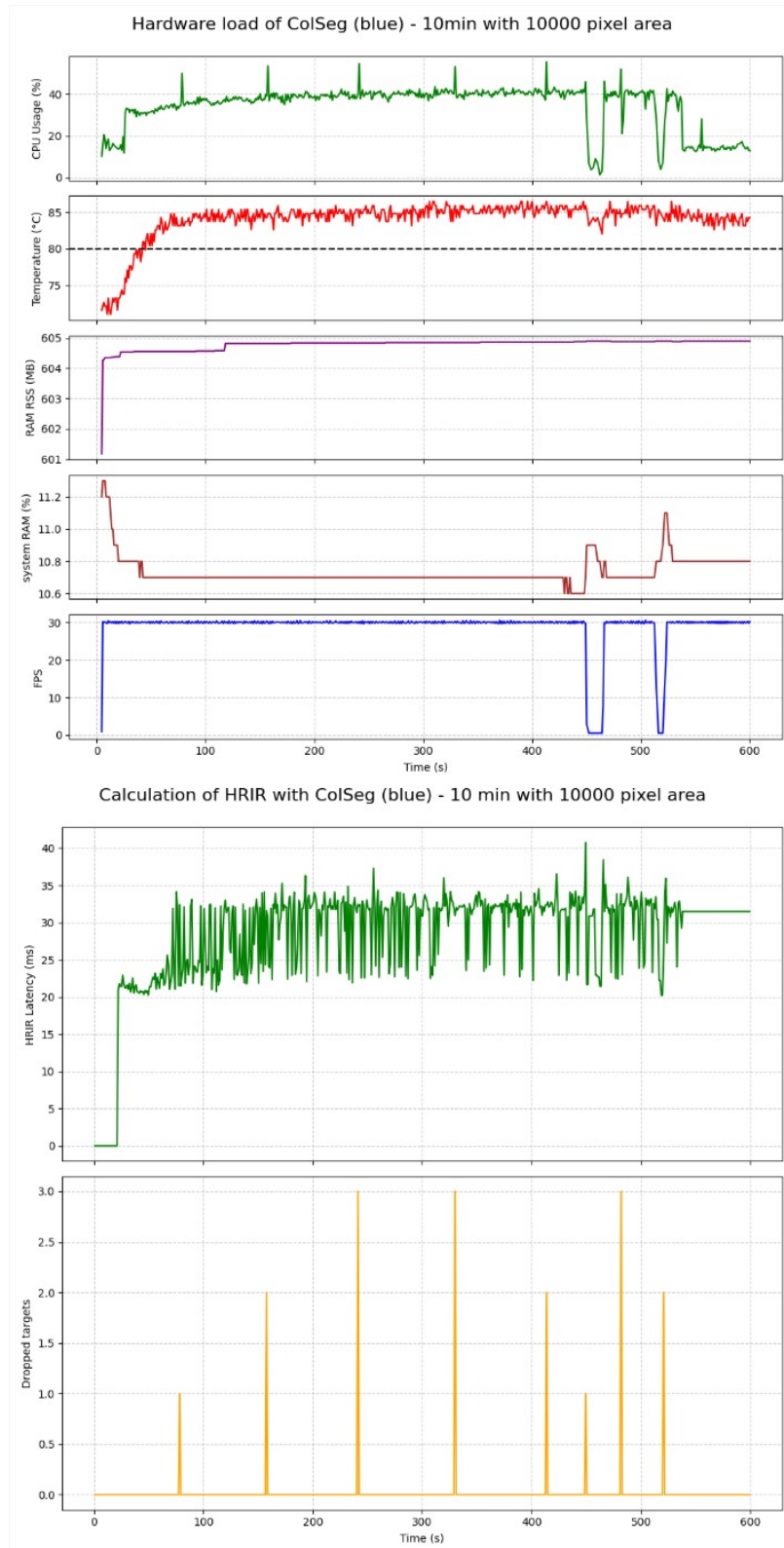


Figure 6.11: System load metrics and HRIR latency using ColSeg for 10 minutes. The minimum valid pixel area for an object was increased to 10,000. The graphs show the following metrics from top to bottom: CPU usage (%), Temperature (°C), RSS (MB), RAM (%), FPS, HRIR latency, and dropped targets.

Table 6.11: Mean system load metrics using ColSeg for 10 minute stress test compared to first 10 minutes of the 1 hour stress test.

N (pixel area)	CPU usage (%)	Temperature (°C)	RSS (MB)	RAM (%)	FPS	Frame duration (ms)	Latency (ms)	Dropped targets (per min)	Dropped targets (total)
400	38.78	84.48	604.58	10.55	30.00	33.33	29.32	1.73	17
10,000	34.58	84.03	604.78	10.73	29.31	34.12	28.31	1.73	17

Short duration tests

Figure 6.8 and Table 6.9 show an interesting test case, where ColSeg performs better than MobileNet SSD during the 120 second stress test. In this case, only a human and balloon were the detected objects. The latter experiences no hickups, while the former receives a massive dip in its FPS and spike in its latency at the end. At this point, the person was moving in and out of frame as fast as they could. A bottleneck occurred for MobileNet SSD at the 115-second mark, but no old targets were thrown out.

A phenomena that can be observed in the HRIR latency graph for the ColSeg execution, is that it remains flat at 20ms between 95 to 112 seconds. This is due to the balloon disappearing from frame, so the graphing algorithm simply inputs the last latency time value every second until an object is detected again. In reality, the actual latency drops to zero because the HRIR is not being calculated at all. This is important to keep in mind for the upcoming results.

Long duration tests

Figures 6.9 and 6.10 showcase the systems behavior using MobileNet SSD and ColSeg respectively during a 1 hour stress test. The numerical data for this experiment can be seen in Table 6.10.

As can be seen in Figure 6.9, MobileNet SSD performs remarkably well on the Raspberry Pi. It boasts a consistently low CPU load of approximately 20% for the entire duration, although spiking for short bursts, likely due to the algorithm switching between objects, as this issue was absent in the controlled tests. The system reaches an admittedly high temperature, however rarely going above 85° for much time, which is when the hard thermal throttling starts to kick in for the Raspberry Pi. The absence of thermal throttling is also evident in the overall stability of the other metrics, such as the previously covered CPU usage and the HRIR latency. In the CPU, temperature, and latency graphs it is possible to see the times when no objects were in frame, i.e. for the time spans of $1505 < t < 1530$, $2343 < t < 3041$ and $3193 < t < 3600$ seconds.

For the 1 hour test with ColSeg, the results were remarkably different, see Figure 6.10. During the runtime, the algorithm often switched between objects because the minimum pixel size was set to $N = 400$, a considerably small number. This was by design, as the purpose of the 1 hour test was to test the system under strain. After all, in a real sports match, the object detection algorithm would likely switch quickly between lots of different players wearing the same color.

Its CPU hit 38%, the temperature exceeded 85°, HRIR latency rose up to 40 ms, and 180 targets were dropped, resulting in stuttering audio. The camera was covered two times to see if the audio could be reset to a more stable state, once at $2034 < t < 2183$ and another time at $3104 < t < 3378$ seconds. After uncovering the camera and letting it resume its detection, the sound playback improved for a few seconds before returning to its stuttering state.

To see if the performance of ColSeg could be improved by filtering out smaller objects, a 10 minute long stress test was performed where the pixel area was increased to $N = 10,000$, meaning fewer bounding-boxes would have to be constructed in the loop that chooses the largest. The result of this test can be seen in Figure 6.11, and a comparison of its values with the first 10 minutes of the 1 hour test can be seen in Table 6.11.

For an increased $N = 10,000$, the amount of dropped targets remained unchanged, but the sound playback was more stable. The mean CPU usage decreased, and the temperature lowered slightly. However, the CPU was affected by two significant drops at approximately 450 and 520 seconds, coinciding with unusual dips close to zero in the FPS, coupled with increased system RAM.

The common denominator for all of these cases was the behavior of the RSS and system RAM. The RSS was as close to a straight line as possible, increasing 0.5 MB in the span of an hour, which is a negligible increase. This means that the code written for both directional perception algorithms does not suffer from memory leaks. Likewise, the flat system RAM curve indicates that memory management during runtime of the algorithms is as close to flawless as can be.



7 Discussion

The chapter discusses the results of the project showcased in chapter 6. The methodology used throughout the thesis is also discussed, along with the ethical and societal implications of the developed system.

7.1 Results

The results of the thesis are discussed in this section.

7.1.1 Calibration

The calibration results for both lenses are discussed here.

Calibration with the default lens

The overfitting of the default lens was likely caused by a combination of the resulting distortion coefficient k_1 , see Appendix A.4, and the images making up the data set. Unfortunately, the chessboard was centralized in too many of the images, giving the model little data on how the chessboard corners are supposed to look when positioned at the edges and corners of the image, where the distortion is the strongest. This combined with the fact that the AI Cameras distortion is relatively subtle, as observed in the "before" images in Figures 4.2 and 6.1, means that the distortion polynomial overfit the known center of the image and warped the relatively unknown edges.

k_1 is the second prime culprit here because $k_1 > -k_2 \Rightarrow k_1 r^2 + k_2 r^4 > 0 \Rightarrow h(r) > 1, r > 0$ across all data splits. The model therefore (incorrectly) identifies the radial distortion to be of the negative (pincushion) sort, hence compensating for it by bulging out the image.

Another piece of evidence that points towards overfitting is the noticeable gap between the RMSEs and mean errors for both lenses. It indicates the presence of outliers in the form of unmodeled lens distortion at edges of the frame and other smaller image imperfections such as glare.

The calibration for the default lens can therefore be considered sub-optimal. However, the intrinsic parameters extracted from the calibration are required for the directional perception, and using an overfitted calibration is better than using none at all. Furthermore, the distortion

brought upon by the overfitting is exceptionally small. It is only visible to the naked eye when the camera captures an image close to a surface with straight lines, so the potential problems it can cause for the directional perception are most likely not significant enough to be noticed by a user of the system. After all, the resulting intrinsic parameters were reasonable, with the optical centers being just a couple of pixels off from the true pixel center of the image, and the focal lengths being close to identical.

Calibration with the wide angle lens

For the calibration with the 110° wide lens, $k_1 < -k_2 \Rightarrow k_1 r^2 + k_2 r^4 < 0 \Rightarrow h(r) < 1, r > 0$ for all data splits. Hence, all of the models correctly identified a positive (barrel) distortion and subsequently applied the opposite operation on the image, effectively straightening out the bulged lines.

Despite the wide angle lens being too heavy and unstable to be usable for the directional perception, the results of the calibration with the lens attachment are important all the same, as they show that the developed calibration algorithm works on wider lenses attached to the AI Camera. This opens up the possibility for future projects where lens attachments adapted for the AI Camera can be used to make the cameras FOV more closely resemble that of a human eye.

Other lenses from the collection in Figure 2.3 with wider FOVs were tried, but the idea of calibrating with them on was quickly dismissed. Their FOVs were so wide that the inner physical edges of the lens attachments could be seen in images captured by the camera. The black edges would have to be considered "dead" pixels, discarding large portions of the frame. The reason why the lens attachments were so ill adapted to the AI Camera was because they were meant to be fit over a standard smartphone camera, the lens of which is significantly larger.

Calibration with exceptionally wide lenses, or even fisheye lenses, is possible. However, investigating a wide assortment of lens attachments which are not suitable for directional perception was not within the scope of this project. Rather, exploring the possibility of calibrating the AI Camera with a wider lens attachment in general was the goal, and it was achieved with a successful calibration at that.

7.1.2 MobileNet SSD vs ColSeg

As mentioned in section 6.2.1, MobileNet used less of the CPU than ColSeg on average. This is not surprising, as the heavier image processing operations are performed mainly by the NN loaded on the IMX500 sensor. ColSeg performs calculations on the image array in all cases, even when no objects are present.

During the 1 hour ColSeg test, CPU usage sat at around 38%, almost double that of MobileNet. As mentioned previously, this is likely caused by the fact that ColSeg performs the heavy image processing computations natively on the Raspberry Pi, as opposed to the IMX500 image sensor taking the load. This can also be seen in the considerably higher temperature of the system, comfortably reaching values of 85°C and upward due to the spiking CPU usage. This was enough to make the Raspberry Pi thermal throttle trigger, underclocking the processor as to avoid overheating, the direct consequence of which can be seen in the HRIR latency spiking to values over 30 ms. The decreased processing power hampers the HRIR execution time, meaning it cannot finish its calculations within the strict deadline of the sound card buffer, resulting in the sound playback stuttering and becoming almost unintelligible over time. Simultaneously, this delay causes a buildup of old target points in the data pipe, which then have to be drained as seen in the bottom graph of Figure 6.10. Meanwhile, the directional perception process keeps running at a stable 30 FPS, consistently sending new target points over the pipeline. By the time all of the old targets have been drained, the newest

target is in a completely different place in terms of its bearing, resulting in the spatial audio skipping discontinuously rather than panning smoothly.

There were a total of three unexpected drops in performance occurring across all tests. Two of them occurred during the 10 minute stress test on ColSeg described in section 6.2.2. The possibility of a memory leak or computational overload is ruled out by the stable RSS and immediate recovery of the system RAM. The main cause is rather the hardware stalling due to the high temperature, causing the CPU to go idle and give a temporary increase in the system RAM buffer. This conclusion is based on previous results of the system underclocking the CPU because of the high temperature when using ColSeg. Given the fact that ColSeg has been confirmed to significantly drive up the temperature of the Pi, and that sound stutters are generally quite common in this method, it is safe to conclude that this problem has a higher risk of recurring here compared to MobileNet SSD.

The third performance drop was the only bottleneck that MobileNet SSD suffered from, see Figure 6.8. Despite the bottleneck, no old targets were thrown out as a result. This was likely due to the system shutting down shortly after, as the code is structured in a way as to check the pipe for the "quit" interrupt first, before checking for older targets, see Figure 5.1. A possible cause for the bottleneck is the CPU going idle, perhaps from a brief overheating, as it resembles the behavior of the system during the 10 minute ColSeg test. Since this was 1 of 9 MobileNet SSD tests that had a bottleneck occur, and the algorithm had no dropped targets across any other tests, it is reasonable to conclude that this is not a problem with a high risk of recurrence.

While MobileNet SSD proved highly reliable for both directional perception and its technical performance, qualitative observations during formative testing indicated severe limitations regarding its depth estimation accuracy at longer ranges. The current depth mapping implemented in the system relies heavily on the pixel footprint of the detected object. While the ColSeg algorithm produced seemingly stable depth estimations due to the rigid geometry and consistent pixel area of the target (a spherical balloon), MobileNet's depth estimations became highly inaccurate as the real-world depth increased, especially near the limit of its detection range. This is an inherent flaw of using bounding-box algorithms on articulating objects. As a human target moves, turns, or becomes partially occluded, the dimensions of the bounding-box fluctuate, changing the depth value. Consequently, MobileNet often registered false depths that were visibly inaccurate compared to the physical environment.

7.2 Method

The section analyses the methodology used throughout the project described in Chapter 4.

7.2.1 Calibration reliability

As mentioned previously in section 7.1.1 and 6.1, the calibration with the default lens was slightly overfitted. It is still reliable for the context of this project, as the extracted intrinsic parameters are sound. But if one wants to conduct more practical observations and to test the limits of the object detection, a better camera model is preferred.

The best option would be to create a new data set of 20 images, focusing on positioning the chessboard in the corners of the frame and at different ranges. Basically, each image should be as varied as possible, with perhaps only one or two where the chessboard is centered. Increasing the amount of images is not entirely out of the question either, if the relevance, diversity, and quality of each image can be guaranteed. If the model still becomes overfitted despite the change to the data set, one of the parameters k_1 or k_2 could potentially be removed.

The calibration for the 110° is more reliable. It managed to successfully model the distortion present and promptly undistort a new image. The undistortion operation should also be tested on an image which does not include the chessboard, such as a lined notebook or any

object with straight lines. But since the model correctly identifies the distortion, it would not be entirely unreasonable to assume that the model would work on any new image.

7.2.2 Directional perception and sound playback

The bearing equations worked as expected, giving accurate angles based on the location of the object in the frame, which could be surmised during formative testing described in section 4.5.2.

However, the methodology chosen for calculating the depth revealed significant limitations depending on the algorithm used. This is despite the users displaying accurate depth perception with their eyes closed in the controlled formative tests thanks to the frequency attenuation. As mentioned before, this is caused by how bounding boxes change depending on the orientation and geometry of the object. More specifically, the current depth calculation relies on each objects real dimensions being known and constant from the cameras perspective. Therefore, the methodology used to obtain the depth is flawed for the context of dynamic PA and sports environments where people jump, crouch, move around, varying in distance from the camera, sometimes not being fully visible, at which point the calculation fails to produce an accurate distance measurement. The same problem would probably appear for ColSeg if a person wore monochromatic clothes as part of a team, as the real-world dimensions would have to be adjusted according to the person, mimicking the bounding box behavior of MobileNet SSD. The current depth calculation is most accurate when the object is of the smaller and more homogeneous variety, such as a sport ball or balloon.

The methodology for obtaining concurrent sound playback alongside the main directional perception process was successful and a critical implementation to assure the systems survival under load. As soon as development of the sound playback began, it was clear that a separate process was the optimal solution. Through separation of the processes, the camera was able to maintain a steady frame rate of 30 FPS even as the audio process bottlenecked due to thermal throttling.

7.2.3 Reproducibility of the method

The general methodology is highly reproducible, provided access to all of the hardware and open source software libraries used in the project. The behavior of MobileNet SSD is deterministic, as it has already been trained and is ready to use. Finally, reproducing the methodology for the calibration is also possible, as the calibration data set is custom. However, there are environmental and physical factors that limit exact quantitative reproduction.

The ColSeg algorithm relies on pre-defined HSV color thresholds, meaning that directional perception results may vary drastically depending on the lighting conditions in the test environment and white-balance settings of the camera.

The reproducibility of specific system load metrics, especially temperature, is heavily dependent on thermal conditions in the environment. If one wants to reproduce the same failure states in terms of thermal throttling, one has to use the exact same passive cooling as in this project. For a description of the environment used for the tests, see section 4.5.2.

While reproduction of the calibration methodology itself is possible, it is practically impossible to reproduce the exact intrinsic parameters and distortion coefficients as the ones in this thesis, as all of the environmental factors and angles from which the chessboard was captured have to be identical to the ones in the calibration data set created for this thesis.

7.3 The work in a wider context

The section discusses the ethical and social implications of creating a technical platform for directional perception of objects in images.

7.3.1 Ethical and societal implications

As outlined in section 1.2 at the start of this thesis, the project aimed to explore principles within a system that could ultimately increase the autonomy of visually impaired people in the context of PA and sports. While the system still has a long way to go in terms of development and stabilization before being reliable in such a context, I believe that the fruits of this project have resulted in a foundational prototype that uses computer vision and machine learning as part of a non-invasive assistive tool. The societal implications of advancing directional perception technology in this context means promoting autonomy and inclusivity for marginalized groups. However, from an ethical perspective, it is important to take user safety into account. As demonstrated, the system exhibits bottlenecks in the form of high temperatures and dropped targets, coupled with inaccurate depth estimations. These are critical engineering challenges that must be solved to ensure that future users are not potentially placed in physical danger due to system latency or hardware failure.



8 Conclusion

The main goal of this thesis project was to develop a prototypical directional perception platform for sonification of objects in images, and to investigate and compare two different object detection methods in this context - MobileNet SSD and ColSeg.

Research question 1 was answered by applying the pinhole camera model on the AI Camera and calibrating it, consequently utilizing the intrinsic camera parameters and a distance formula to successfully and accurately translate a 2D pixel point to a 3D target vector. This was achieved despite central clustering of the calibration data leading to slight overfitting. For cases when a wider FOV is desired, it was proven that the AI camera could be successfully calibrated with a wide angle lens attached. Ultimately, a properly conducted calibration determines how well digital pixels are translated into accurate coordinates for sonification.

Research question 2 was answered by choosing the largest bounding box over a strict confidence threshold, combined with resolution downscaling for ColSeg, followed by extraction of the bounding box 2D middle pixel point, coordinate mapping, and subsequent calculation of the bearing using trigonometry. For ColSeg in the context of real time sonification, downscaling is mandatory to ensure stable performance.

Research question 3 was answered mostly through formative practical tests. Both algorithms give an accurate bearing to the object, and the depth calculation combined with attenuation of the frequency gave users a solid perception that worked well in practical tests. However, the quantitative distance estimations produced by MobileNet SSD were highly unreliable. ColSeg works better for homogenous objects such as balloons, but would likely also struggle with less rigid objects. Therefore, a reliable tool for directional perception for visually impaired individuals should preferably rely on more accurate, hardware-based distance estimation techniques.

Research question 4 was answered through a collection of system bottleneck and performance tests. MobileNet SSD greatly benefited in its performance on the Raspberry Pi by delegating its image processing to the IMX500 sensor on the AI camera module. During the 1 hour test, the system maintained a stable 20% CPU load and had no target drops over an hour. On the contrary, the heavy computations performed within the ColSeg algorithm caused thermal throttling at 85°C, cascading into HRIR calculation delays, buffer backlogs, and severe audio stuttering. For real-time sonification applications, freeing up the host CPU exclusively for HRIR calculations is paramount for long term system survivability.

8.1 Future work

Future iterations of this platform should upgrade and include a thorough ground-truth evaluation of the depth calculation, along with an investigation of the drop-off distances for when an object disappears out of range of the object detection algorithm.

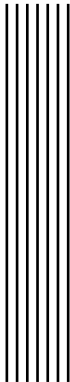
Alternatively, depth perception can be achieved through upgrades in the hardware, such as a stereo camera or LiDAR, which would bypass the reliance on the bounding-boxes for distance estimation.

To reduce the CPU load and compensate for the thermal bottleneck, one could create a lookup table containing pre-computed HRIRs at different bearings and depths, making a "sphere" of points. The temperature could also be regulated by attaching a cooling fan on the Pi.

Tests in more varied environments could be conducted, such as a large indoor or outdoor court, as well as under different lighting conditions. Eventually, it would be appropriate to test the system on users who have visual impairments, letting them navigate obstacle courses with upright poles, for instance.

Since the AI camera can be loaded with a custom NN, it would be interesting and suitable to train a NN specifically on sports equipment, since MobileNet SSD is also trained on redundant objects such as planes and boats. The color segmentation could also be upgraded to utilize oval fitting to capture a more accurate representation of the objects dimensions, rather than through rectangular bounding-boxes, if hardware stability can be assured.

Finally, if the system is safe enough, the platform itself can be made portable and wearable. A power bank can be used as a power source, as imagined in the original system illustration, and the object localization script could be run on startup. A small controller with buttons can be used to choose between different object detection algorithms and control the system.



Bibliography

- [1] Christer Malm, Johan Jakobsson, and Andreas Isaksson. “Physical Activity and Sports—Real Health Benefits: A Review with Insight into the Public Health of Sweden”. In: *Sports* 7.5 (2019). ISSN: 2075-4663. DOI: 10.3390/sports7050127.
- [2] Laura Mandolesi, Arianna Polverino, Simone Montuori, Francesca Foti, Giampaolo Ferraioli, Pierpaolo Sorrentino, and Giuseppe Sorrentino. “Effects of Physical Exercise on Cognitive Functioning and Wellbeing: Biological and Psychological Benefits”. In: *Frontiers in Psychology* 9 (2018). ISSN: 1664-1078. DOI: 10.3389/fpsyg.2018.00509.
- [3] Fernando Gomez-Pinilla and Charles Hillman. “The influence of exercise on cognitive abilities”. In: *Compr Physiol* 3 (2013), pp. 403–428. DOI: 10.1002/j.2040-4603.2013.tb00485.x.
- [4] *Blindness and vision impairment*. Feb. 10, 2026. URL: <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment> (visited on 04/22/2026).
- [5] *Adaptive Equipment*. URL: <https://britishblindsport.org.uk/equipment> (visited on 03/05/2026).
- [6] Alexandra R. Jamieson and Helara D. Wijesundara. “A review of adaptive equipment and technology for exercise and sports activities for people with disabilities”. In: *Disability and Rehabilitation: Assistive Technology* 20.1 (2025), pp. 33–45. DOI: 10.1080/17483107.2024.2372323.
- [7] *Para Alpine Skiing*. URL: <https://www.paralympic.org/alpine-skiing> (visited on 04/27/2026).
- [8] Edward R. Laskowski. “Snow Skiing for the Physically Disabled”. In: *Mayo Clinic Proceedings* 66.2 (1991), pp. 160–172. ISSN: 0025-6196. DOI: [https://doi.org/10.1016/S0025-6196\(12\)60488-1](https://doi.org/10.1016/S0025-6196(12)60488-1).
- [9] Juan Haladjian, Maximilian Reif, and Bernd Brügge. “VIHapp: a wearable system to support blind skiing”. In: *Proceedings of the 2017 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2017 ACM International Symposium on Wearable Computers*. UbiComp ’17. Maui, Hawaii: Association for Computing Machinery, 2017, pp. 1033–1037. ISBN: 9781450351904. DOI: <https://doi.org/10.1145/3123024.3124443>.

-
- [10] *AI Camera*. URL: <https://www.raspberrypi.com/documentation/accessories/ai-camera.html> (visited on 05/04/2026).
 - [11] Simon Hansson. "Algorithms for Sonifying Objects into Spatial Audio using Head-Related Transfer Functions". 30 ECTS. Master's thesis. Linköping, Sweden: Linköping University, Department of Electrical Engineering, 2026.
 - [12] *Advanced Linux Sound Architecture*. URL: https://wiki.archlinux.org/title/Advanced_Linux_Sound_Architecture (visited on 05/18/2026).
 - [13] V.R. Algazi, R.O. Duda, D.M. Thompson, and C. Avendano. "The CIPIC HRTF database". In: *Proceedings of the 2001 IEEE Workshop on the Applications of Signal Processing to Audio and Acoustics (Cat. No.01TH8575)*. 2001, pp. 99–102. DOI: 10.1109/ASPAA.2001.969552.
 - [14] N. Agrawal, A. Kumar, Varun Bajaj, and G.K. Singh. "Design of digital IIR filter: A research survey". In: *Applied Acoustics* 172 (2021), p. 107669. ISSN: 0003-682X. DOI: <https://doi.org/10.1016/j.apacoust.2020.107669>.
 - [15] *multiprocessing — Process-based parallelism*. URL: <https://docs.python.org/3/library/multiprocessing.html> (visited on 05/20/2026).
 - [16] Stuart Russel and Peter Norvig. *Artificial Intelligence: A Modern Approach, 4th Global ed.* Prentice Hall, 2020. Chap. 21.
 - [17] *python*. URL: <https://www.python.org/> (visited on 05/18/2026).
 - [18] Lutz Bornmann. "The sound of science : Data sonification has emerged as possible alternative to data visualization". In: *EMBO reports* 25.9 (2024), pp. 3743–3747. DOI: 10.1038/s44319-024-00230-6.
 - [19] Raspberry Pi Ltd. *Raspberry Pi 5*. English. Raspberry Pi Ltd. 2026. 6 pp.
 - [20] INNOMAKER. *Innomaker DAC /DAC Mini HAT UserManual*. English. INNOMAKER. 2024. 49 pp.
 - [21] Raspberry Pi Ltd. *Raspberry Pi AI Camera*. English. Raspberry Pi Ltd. 2024. 6 pp.
 - [22] *Sony's IMX500*. URL: <https://www.aitrios.sony-semicon.com/edge-ai-devices/imx500> (visited on 05/18/2026).
 - [23] Tomasz Mazuryk and Michael Gervautz. *Virtual Reality: History, Applications, Technology and Future*. Tech. rep. TR-186-2-96-06. Vienna, Austria: Institute of Computer Graphics, Vienna University of Technology, Feb. 1996.
 - [24] Geethanjali Prakash, Metun Nivin, and M. Rajeswari. "Advances in Ecological Surveillance: Real-Time Wildlife Detection using MobileNet-SSD V2 Convolutional Neural Network". In: *International Journal for Research in Applied Science and Engineering Technology* Volume 11 (Dec. 2023), p. 15. DOI: 10.22214/ijraset.2023.57847.
 - [25] H.D. Cheng, X.H. Jiang, Y. Sun, and Jingli Wang. "Color image segmentation: advances and prospects". In: *Pattern Recognition* 34.12 (2001), pp. 2259–2281. ISSN: 0031-3203. DOI: [https://doi.org/10.1016/S0031-3203\(00\)00149-7](https://doi.org/10.1016/S0031-3203(00)00149-7).
 - [26] Felix Järemo Lawin. "Depth data processing and 3D reconstruction using the Kinect v2". 30 ECTS. Master's thesis. Linköping, Sweden: Linköping University, Department of Electrical Engineering, 2015.
 - [27] Klas Nordberg. *Introduction to Representations and Estimation in Geometry*. Linköping University Electronic Press, 2018. Chap. 18.
 - [28] *Radial Distortion*. URL: <https://www.sciencedirect.com/topics/computer-science/radial-distortion> (visited on 04/07/2026).

-
- [29] Yue Liu, Dewen Cheng, Qiwei Wang, Qichao Hou, Luo Gu, Hailong Chen, Tong Yang, and YongTian Wang. "Optical distortion correction considering radial and tangential distortion rates defined by optical design". In: *Results in Optics* 3 (2021), p. 100072. ISSN: 2666-9501. DOI: <https://doi.org/10.1016/j.rio.2021.100072>.
- [30] Duane C. Brown. "Decentering Distortion of Lenses". In: *Photogrammetric Engineering* (May 1966), pp. 444–462.
- [31] Z. Zhang. "A flexible new technique for camera calibration". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.11 (2000), pp. 1330–1334. DOI: 10.1109/34.888718.
- [32] Dany Eka Saputra, Aloysius Shendy Mulyana Senjaya, Joshua Ivander, and Alex William Chandra. "Experiment on Distance Measurement Using Single Camera". In: *2021 4th International Conference on Information and Communications Technology (ICOIACT)*. 2021, pp. 80–85. DOI: 10.1109/ICOIACT53268.2021.9564010.
- [33] L. Nielsen, M. Mahowald, and C. Mead. *SeeHear*. Technical Report TFRT-7355. Lund Institute of Technology (LTH), 1987.
- [34] M. Naemura, A. Fukuda, Y. Mizutani, Y. Izumi, Y. Tanaka, and K. Enami. "Morphological segmentation of sport scenes using color information". In: *IEEE Transactions on Broadcasting* 46.3 (2000), pp. 181–188. DOI: 10.1109/11.892154.
- [35] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. "You Only Look Once: Unified, Real-Time Object Detection". In: *arXiv preprint arXiv:1506.02640* (2016).
- [36] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. "MobileNetV2: Inverted Residuals and Linear Bottlenecks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2018.
- [37] *OpenCV modules*. URL: <https://docs.opencv.org/4.x/index.html> (visited on 05/03/2026).
- [38] *Camera Calibration*. URL: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html (visited on 04/07/2026).



Appendix

A.1 Terminal commands

A.1.1 Display video feed

```
$ rpicam-hello --timeout 0
```

A.1.2 Capture image

```
$ rpicam-jpeg --output image00xx.jpg --timeout 3000
```

A.2 Code

A.2.1 Calibration template

```
import numpy as np
import cv2 as cv
import glob

# termination criteria
criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....., (6,5,0)
objp = np.zeros((6*7,3), np.float32)
objp[:, :2] = np.mgrid[0:7,0:6].T.reshape(-1,2)

# Arrays to store object points and image points from all the images.
objpoints = [] # 3d point in real world space
imgpoints = [] # 2d points in image plane.

images = glob.glob('*.jpg')
```

```
for fname in images:
    img = cv.imread(fname)
    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

    # Find the chess board corners
    ret, corners = cv.findChessboardCorners(gray, (7,6), None)

    # If found, add object points, image points (after refining them)
    if ret == True:
        objpoints.append(objp)

        corners2 = cv.cornerSubPix(gray,corners, (11,11), (-1,-1), criteria)
        imgpoints.append(corners2)

        # Draw and display the corners
        cv.drawChessboardCorners(img, (7,6), corners2, ret)
        cv.imshow('img', img)
        cv.waitKey(500)

cv.destroyAllWindows()

ret, mtx, dist, rvecs, tvecs = cv.calibrateCamera(objpoints, imgpoints,
    gray.shape[::-1], None, None)
```

A.2.2 Undistortion template

```
img = cv.imread('left12.jpg')
h, w = img.shape[:2]
newcameramtx, roi = cv.getOptimalNewCameraMatrix(mtx, dist, (w,h), 1,
    (w,h))

# undistort
dst = cv.undistort(img, mtx, dist, None, newcameramtx)

# crop the image
x, y, w, h = roi
dst = dst[y:y+h, x:x+w]
cv.imwrite('calibresult.png', dst)
```

A.2.3 Re-projection template

```
mean_error = 0
for i in range(len(objpoints)):
    imgpoints2, _ = cv.projectPoints(objpoints[i], rvecs[i], tvecs[i], mtx,
        dist)
    error = cv.norm(imgpoints[i], imgpoints2, cv.NORM_L2SQR) /
        len(imgpoints2)
    mean_error += error

print( "total error: {}".format(np.sqrt(mean_error/len(objpoints))) )
```

A.2.4 Re-routing audio

```
pcm.!default{
    type plug
    slave.pcm "hw:2,0"
}

ctl.!default{
    type hw
    card 2
}

```

A.3 Tables

A.3.1 Calibration errors for all data splits

Table A.1: Errors from the calibration with a 60/40 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.095	0.112	1.205	1.254
2	0.088	0.125	1.001	1.558
3	0.106	0.101	1.313	1.119
4	0.096	0.114	1.195	1.273
5	0.086	0.125	0.983	1.538

Table A.2: Errors from the calibration with a 70/30 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.106	0.092	1.293	1.009
2	0.085	0.134	0.979	1.705
3	0.106	0.087	1.290	0.971
4	0.106	0.095	1.301	1.029
5	0.107	0.091	1.306	1.006

Table A.3: Errors from the calibration with a 80/20 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.098	0.115	1.190	1.292
2	0.106	0.089	1.269	0.989
3	0.099	0.110	1.213	1.210
4	0.104	0.088	1.264	0.999
5	0.105	0.091	1.267	0.989

Table A.4: Errors from the calibration with a 90/10 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.100	0.099	1.220	1.067
2	0.102	0.085	1.236	0.914
3	0.088	0.192	1.007	2.305
4	0.101	0.095	1.220	1.085
5	0.090	0.179	1.022	2.246

A.3.2 Calibration errors for all data splits with a wide angle lens

Table A.5: Errors from the calibration with the attached 110° lens with a 60/40 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.345	0.327	4.024	3.744
2	0.312	0.382	3.613	4.199
3	0.325	0.376	3.802	4.303
4	0.322	0.375	3.733	4.231
5	0.308	0.383	3.375	4.753

Table A.6: Errors from the calibration with the attached 110° lens with a 70/30 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.287	0.403	3.405	4.418
2	0.262	0.473	3.0135	5.432
3	0.317	0.373	3.685	4.183
4	0.318	0.381	3.606	4.590
5	0.373	0.283	4.270	3.127

Table A.7: Errors from the calibration with the attached 110° lens with a 80/20 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.300	0.40366	3.5269	4.462355
2	0.3239	0.34268	3.77030	3.70806
3	0.325075	0.3419692	3.76988	3.6989
4	0.300764	0.47240	3.37457	5.80424
5	0.304883	0.46435	3.54569	5.07317

Table A.8: Errors from the calibration with the attached 110° lens with a 90/10 split.

Test	Re-projection	Mean validation	Re-projection RMSE	Validation RMSE
1	0.325	0.342	3.770	3.699
2	0.332	0.295	3.843	3.268
3	0.284	0.652	3.240	7.071
4	0.334	0.230	3.844	2.483
5	0.313	0.504	3.610	5.450

A.4 Calibration results with default lens

A.4.1 60/40 split

$$d_c = (0.03815, -0.004646, 0.00000085719, -0.001627, 0) \quad (\text{A.1})$$

$$\mathbf{C} = \begin{bmatrix} 3080.517 & 0 & 2024.294 \\ 0 & 3078.620 & 1521.126 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.2})$$

A.4.2 70/30 split

$$d_c = (0.04173, -0.01311, -0.000004857, -0.001196, 0) \quad (\text{A.3})$$

$$\mathbf{C} = \begin{bmatrix} 3075.401 & 0 & 2027.016 \\ 0 & 3072.120 & 1521.795 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.4})$$

A.4.3 80/20 split

$$d_c = (0.03765, 0.002720, -0.000888, -0.0001855, 0) \quad (\text{A.5})$$

$$\mathbf{C} = \begin{bmatrix} 3077.529 & 0 & 2033.227 \\ 0 & 3074.940 & 1519.129 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.6})$$

A.4.4 90/10 split

$$d_c = (0.04216, -0.01535, -0.0002655, -0.0001409, 0) \quad (\text{A.7})$$

$$\mathbf{C} = \begin{bmatrix} 3073.318 & 0 & 2036.559 \\ 0 & 3070.779 & 1523.781 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.8})$$

A.5 Calibration results with wide lens

A.5.1 60/40 split

$$d_c = (-0.095232, 0.033684, -0.005754, -0.0001256, 0) \quad (\text{A.9})$$

$$\mathbf{C} = \begin{bmatrix} 2059.365 & 0 & 2100.917 \\ 0 & 2064.329 & 1392.282 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.10})$$

A.5.2 70/30 split

$$d_c = (-0.082452, 0.014561, -0.004863, -0.0006466, 0) \quad (\text{A.11})$$

$$\mathbf{C} = \begin{bmatrix} 2048.289 & 0 & 2088.331 \\ 0 & 2049.370 & 1373.807 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.12})$$

A.5.3 80/20 split

$$d_c = (-0.080323, 0.011612, -0.005716, -0.0002528, 0) \quad (\text{A.13})$$

$$\mathbf{C} = \begin{bmatrix} 2030.216 & 0 & 2087.646 \\ 0 & 2030.043 & 1382.989 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.14})$$

A.5.4 90/10 split

$$d_c = (-0.077817, 0.007861, -0.005603, 0.00002476, 0) \quad (\text{A.15})$$

$$\mathbf{C} = \begin{bmatrix} 2034.529 & 0 & 2095.130 \\ 0 & 2035.916 & 1379.670 \\ 0 & 0 & 1 \end{bmatrix} \quad (\text{A.16})$$